

Tema 2

JavaScript



De dónde viene JavaScript, cómo se prueba lo primero que se escribe con él, y el vocabulario básico (variables, tipos, conversión, operadores y comentarios) con el que se construye cualquier programa.

1. Fundamentos de JavaScript

1.1 Introducción

JavaScript es, hoy en día, uno de los lenguajes de programación más utilizados del mundo, por tres razones muy concretas: es útil (resuelve problemas reales de interacción con el usuario), es práctico (no requiere instalación ni compilación previa) y está disponible en cualquier navegador web, en cualquier sistema operativo, sin que el usuario tenga que hacer nada especial para ejecutarlo.

Un poco de historia

JavaScript fue creado por Brendan Eich en 1995, mientras trabajaba en Netscape Communications, la empresa responsable de uno de los primeros navegadores web populares. El lenguaje se diseñó y se implementó en muy pocos días, con un objetivo modesto: añadir pequeñas dosis de interactividad a páginas web que, hasta entonces, eran completamente estáticas.

Durante su desarrollo pasó por varios nombres: primero se llamó Mocha, después LiveScript, y finalmente se rebautizó como JavaScript poco antes de su lanzamiento (una decisión de mercado, aprovechando la popularidad que tenía Java en ese momento, aunque ambos lenguajes son técnicamente muy distintos entre sí y no tienen relación directa). Lo que nació como un lenguaje sencillo y secundario ha terminado convirtiéndose en el lenguaje de programación que se ejecuta en prácticamente todos los sitios web del mundo.

Características principales

- **Ligero:** su sintaxis y su motor de ejecución están pensados para ser rápidos de cargar e interpretar en el navegador.
- **Multiplataforma:** se ejecuta igual en Windows, Linux o macOS, y en cualquier navegador que lo implemente, sin necesidad de adaptar el código.
- **Basado en prototipos:** a diferencia de lenguajes como Java o C#, JavaScript no usa clases de forma nativa para la herencia, sino un mecanismo llamado *prototipos*.
- **Orientado a objetos y basado en eventos:** casi todo en JavaScript es un objeto, y gran parte de la interacción con la página se construye reaccionando a eventos (un clic, una tecla pulsada, una carga completada).
- **Interpretado:** no requiere un paso de compilación previo por parte del desarrollador antes de ejecutarse (como ya se vio en el Tema 1, el motor del navegador lo procesa internamente, pero eso es transparente para quien programa).
- **Débilmente tipado** (tipado dinámico): una misma variable puede almacenar valores de distintos tipos a lo largo de la ejecución, sin que haga falta declarar de antemano qué tipo de dato va a contener.

1.2 Primer contacto: Hola Mundo

Antes de entrar en la teoría del lenguaje, conviene ver JavaScript funcionando. Existen varias formas de mostrar un mensaje al usuario o de comprobar que un script se está ejecutando correctamente.

La función alert

Muestra una ventana emergente del propio navegador (no forma parte de la página) con un mensaje y un botón de aceptar. Detiene la ejecución del script hasta que el usuario la cierra.

```
alert('Hola mundo');
```

El método write del objeto document

Escribe texto directamente en el documento HTML, en el punto donde se ejecuta el script.

```
document.write('Hola mundo');
```

Importante: document.write() se enseña aquí porque aparece con frecuencia en tutoriales antiguos, pero está desaconsejado en desarrollo profesional: si se llama después de que la página haya terminado de cargar, borra por completo el contenido del documento y lo sustituye por el texto escrito. Se muestra únicamente con fines didácticos, para reconocerlo si aparece en código de otros.

El objeto console

Envía el mensaje a la consola de las herramientas de desarrollador del navegador, sin interrumpir ni modificar la página. Es la herramienta que se usará de forma constante a lo largo de todo el curso, tanto para mostrar resultados como para depurar código.

```
console.log('Hola mundo');
```

Nota: De las tres opciones, console.log() es la que se usará por defecto en el resto del curso: no interrumpe la ejecución con una ventana emergente, no modifica el documento, y es la herramienta estándar de depuración en cualquier entorno de desarrollo profesional.

Actividad · Hola Mundo

Crea tu primer fichero .html con un <script> dentro de la etiqueta <body> que muestre "Hola mundo". Prueba, uno por uno, los tres métodos vistos (alert, document.write y console.log) y observa las diferencias.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Hola mundo</title>
</head>
<body>
  <h1>Mi primer script</h1>

  <script>
    console.log('Hola mundo');
  </script>
</body>
</html>
```

1.3 La etiqueta noscript

A veces el navegador no puede interpretar JavaScript: bien porque el navegador no lo soporta, bien porque el propio usuario lo ha desactivado desde su configuración. En esos casos, la página se carga igualmente, pero todo el comportamiento que dependía de JavaScript deja de funcionar, sin ningún aviso.

Por esa razón, es buena práctica avisar de que la página necesita JavaScript para funcionar correctamente, usando la etiqueta `<noscript>`. Su contenido solo se muestra cuando el navegador no ejecuta JavaScript; en caso contrario, se ignora por completo.

```
<noscript>
  <p>La página que estás viendo requiere JavaScript para su correcto
funcionamiento.</p>
</noscript>
```

1.4 Funciones de interacción: prompt y confirm

JavaScript incluye tres funciones integradas que muestran una pequeña ventana del navegador para comunicarse con el usuario. Ya se ha visto `alert`, aquí se completan las otras dos, muy útiles para empezar a experimentar con datos que introduce el propio usuario.

prompt(mensaje, valorPorDefecto)

Muestra un mensaje, un campo de texto y dos botones (aceptar y cancelar). Devuelve el texto introducido como string, o null si el usuario pulsa "Cancelar".

```
const nombreCliente = prompt('¿Cuál es tu nombre?', '');

if (nombreCliente === null) {
  console.log('El usuario ha cancelado el diálogo');
} else {
  console.log(`Hola, ${nombreCliente}`);
}
```

confirm(pregunta)

Muestra un mensaje y dos botones (aceptar y cancelar). Devuelve true o false según el botón pulsado.

```
const confirmado = confirm('¿Seguro que quieres eliminar este producto?');

if (confirmado) {
  console.log('Producto eliminado');
} else {
  console.log('Operación cancelada');
}
```

Importante: Las tres funciones son **bloqueantes (también alert)**: mientras la ventana está abierta, toda la página deja de responder, incluida cualquier otra ejecución de JavaScript. Por eso, en aplicaciones web profesionales no se usan en la interfaz final (se sustituyen por ventanas modales propias, construidas con HTML y CSS, que no bloquean el navegador). Aquí se usan porque son la forma más rápida de leer un dato del usuario o mostrar un resultado sin necesitar todavía el DOM, que se estudiará más adelante.

1.5 Enlazar HTML con JavaScript: `<script>` y sus variantes

Hay dos decisiones distintas que se mezclan a menudo: dónde escribes el código (dentro del HTML o en un fichero aparte) y dónde colocas la etiqueta `<script>` (en el `<head>` o en el `<body>`, y con qué atributos). Te lo explico por partes.

Las tres formas de vincular el código

Inline (dentro de un atributo, tipo `onclick="..."`): mala práctica, mezcla HTML y JS. Se enseña para reconocerlo, no para usarlo.

Interno: el código va dentro de la propia etiqueta `<script>`, en el mismo HTML.

```
<script>
  console.log('Hola');
</script>
```

Externo: el código vive en un fichero `.js` aparte, enlazado con el atributo `src`. Es la forma recomendada siempre que el código no sea trivial.

```
<script src="./app.js"></script>
```

Dónde colocar la etiqueta `<script>` (y con qué atributo)

Aquí está el matiz que seguramente te interesa. El navegador parsea el HTML de arriba abajo, y por defecto, en cuanto encuentra un `<script>`, detiene ese parseo para descargar y ejecutar el script antes de seguir.

Colocación	Comportamiento	¿Cuándo usarla?
<code><script src="..."></code> en el <code><head></code> , sin atributos	Bloquea el parseo del HTML nada más encontrarlo. Si el script intenta acceder al DOM (<code>document.querySelector(...)</code>), falla porque el <code><body></code> todavía no existe.	Nunca, para código que toque el DOM
<code><script src="..."></code> al final del <code><body></code> , justo antes de <code></body></code>	Sigue siendo bloqueante, pero como ya está todo el HTML anterior parseado, el DOM ya existe cuando se ejecuta. Es la técnica clásica de "toda la vida".	Válida, pero superada por defer
<code><script defer src="..."></code> en el <code><head></code>	Se descarga en paralelo sin bloquear el parseo, y se ejecuta justo cuando el HTML ya está completo. Respeta el orden entre varios scripts.	La opción recomendada hoy en día
<code><script type="module" src="..."></code> en el <code><head></code>	Se comporta como defer automáticamente, además de dar módulos ES (<code>import/export</code>).	Cuando trabajas con módulos

Por qué defer en el `<head>` es hoy la mejor práctica

La idea clave: con defer, el navegador empieza a descargar el script cuanto antes (nada más leer el `<head>`), pero espera a ejecutarlo hasta que el HTML esté completo.

Consigues lo mejor de los dos mundos: descarga temprana, ejecución segura.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi página</title>
  <script defer src="./app.js"></script>
</head>
```

```
<body>
  <h1 id="titulo"></h1>
</body>
</html>
```

Si en cambio lo pones al final del `<body>` sin `defer`, el navegador ni siquiera empieza a descargarlo hasta que ha terminado de parsear todo el HTML anterior, así que en páginas grandes pierdes ese tiempo de descarga en paralelo que sí aprovechas con `defer`.

1.6 Sintaxis básica del lenguaje

Antes de entrar en detalle en cada elemento del lenguaje, conviene fijar unas pocas reglas generales que se van a cumplir siempre, en cualquier programa JavaScript.

Los espacios en blanco y los saltos de línea no importan

El motor de JavaScript ignora los espacios, tabulaciones y saltos de línea adicionales. Se usan únicamente para que el código sea más legible para las personas, no para el propio motor.

```
// Estas dos formas son equivalentes para el motor de JavaScript
const total = 10 + 5;

const
  total2
  =
    10 + 5;
```

Nota: Que el motor las ignore no significa que a las personas les dé igual. Un formateador como Prettier (mencionado en el Tema 1) se encarga de aplicar un espaciado consistente de forma automática, para que el equipo no tenga que discutir sobre estilo.

Es sensible a mayúsculas y minúsculas (*case sensitive*)

JavaScript distingue mayúsculas de minúsculas en todos sus identificadores: nombres de variables, funciones, propiedades y palabras clave. Dos nombres que solo se diferencien en el uso de mayúsculas son, para el lenguaje, dos cosas completamente distintas.

```
const nombreUsuario = 'ana';
const NombreUsuario = 'ANA';

console.log(nombreUsuario); // 'ana'
console.log(NombreUsuario); // 'ANA' (una variable totalmente distinta a la anterior)
```

Es un lenguaje débilmente tipado

Como se ha adelantado anteriormente, al declarar una variable no se indica de qué tipo será su valor, y ese valor puede cambiar de tipo durante la ejecución del programa.

El punto y coma es opcional, pero muy recomendable

JavaScript no obliga a terminar cada sentencia con punto y coma (;): el motor intenta insertarlos automáticamente donde cree que corresponde, mediante un mecanismo

llamado *ASI (Automatic Semicolon Insertion)*. El problema es que ese mecanismo no siempre acierta, y puede producir errores silenciosos muy difíciles de detectar.

Se pueden incluir comentarios

Con `//` para una sola línea, o con `/* y */` para varias líneas. Más adelante se trata en profundidad, incluyendo el estándar JSDoc usado en desarrollo profesional.

1.7 Variables: declaración, inicialización y ámbito

Una variable es un espacio con nombre donde se almacena un valor que puede cambiar durante la ejecución del programa. JavaScript ofrece tres palabras clave para declarar variables: **var**, **let** y **const**. Aunque **var** sigue siendo válida y aparece en código antiguo, en desarrollo profesional actual no se utiliza.

Declaración e inicialización

Declarar una variable es crear el espacio de memoria y asociarle un nombre. **Inicializar** es asignarle un primer valor. Ambas acciones pueden hacerse por separado o a la vez.

```
// Declaración sin inicializar
let precioProducto;
console.log(precioProducto); // undefined

// Declaración con inicialización en la misma línea
let nombreProducto = 'Teclado mecánico';

// const obliga a inicializar en el momento de la declaración
const IVA = 0.21;

// Esto lanza un error de sintaxis: const exige valor inicial
const stockMinimo;
```

let vs. const: cuándo usar cada una

	let	const
¿Se puede reasignar?	Sí	No
¿Se puede declarar sin valor inicial?	Sí	No
¿Cuándo usarla?	El valor va a cambiar	El valor no se va a reasignar

```
let contadorVisitas = 0;
contadorVisitas = contadorVisitas + 1; // correcto, let permite reasignar

const nombreEmpresa = 'Acme Software';
nombreEmpresa = 'Otra empresa'; // TypeError: Assignment to constant variable.
```

Importante: La recomendación en desarrollo profesional es usar **const** por defecto y recurrir a **let** únicamente cuando el valor vaya a cambiar (por ejemplo, un contador de un bucle o un acumulador). No es solo una cuestión de estilo: declarar algo como **const** documenta la intención del código y evita reasignaciones accidentales, una fuente habitual de errores difíciles de rastrear.

Actividad · Calculadora de propina

Crea un script que pida al usuario, con prompt, el importe de una cuenta de restaurante y, con otro prompt, el porcentaje de propina que quiere dejar. Calcula el total y muéstralo con alert. Añade un confirm antes de mostrar el resultado, preguntando "¿Confirmas el cálculo?"; si el usuario cancela, muestra por consola "Cálculo cancelado" en su lugar.

Ámbito (scope) de las variables

El ámbito determina en qué parte del código una variable es accesible. let y const tienen ámbito de bloque (block scope): solo existen dentro de las llaves { } en las que se declaran.

```
function calcularPrecioFinal(precioBase) {  
  if (precioBase > 100) {  
    const descuento = precioBase * 0.1;  
    console.log(descuento); // accesible: estamos dentro del bloque if  
  }  
  
  console.log(descuento); // ReferenceError: descuento is not defined  
}
```

var, en cambio, tiene **ámbito de función** (function scope), no de bloque, lo que provoca comportamientos poco intuitivos:

```
function calcularPrecioFinalConVar(precioBase) {  
  if (precioBase > 100) {  
    var descuento = precioBase * 0.1;  
  }  
  
  console.log(descuento); // funciona, aunque estamos FUERA del if  
}
```

Importante: Este es uno de los motivos por los que **var** se considera obsoleta: su ámbito de función hace que una variable "se escape" del bloque donde parecía estar confinada, lo cual facilita errores silenciosos en funciones largas. let y const resuelven este problema restringiendo la variable exactamente al bloque { } donde se declara.

Hoisting

JavaScript "eleva" las declaraciones de variables y funciones a la parte superior de su ámbito antes de ejecutar el código, un comportamiento llamado **hoisting**. Sin embargo, let y const quedan en lo que se conoce como *temporal dead zone*: existen, pero no se pueden usar antes de la línea donde se declaran.

```
console.log(mensaje); // ReferenceError: Cannot access 'mensaje' before  
initialization  
let mensaje = 'Hola';
```

Nota: No es necesario memorizar los detalles internos del *hoisting*. A efectos prácticos, la regla de oro es: declara siempre las variables al principio del bloque donde se van a usar, antes de leerlas o modificarlas. Siguiendo esa disciplina, el *hoisting* nunca da sorpresas.

1.8 Tipos de datos

JavaScript es un lenguaje de **tipado dinámico**: no se declara el tipo de una variable, sino que el tipo lo determina el valor que contiene en cada momento, y puede cambiar durante la ejecución.

```
let valor = 42; // valor es de tipo number
valor = 'cuarenta y dos'; // ahora valor es de tipo string
```

JavaScript distingue entre **tipos primitivos** y **tipos por referencia**.

Tipos primitivos

Un valor primitivo es inmutable y se compara y se copia **por valor**.

Tipo	Descripción	Ejemplo
number	Números enteros y decimales (un único tipo numérico)	42, 3.14, -7
string	Cadenas de texto	'Hola', "Hola"
boolean	Verdadero o falso	true, false
undefined	Variable declarada, pero sin valor asignado	let x;
null	Ausencia intencionada de valor	let x = null;
bigint	Enteros de precisión arbitraria	9007199254740993n
symbol	Valor único e irrepetible	Symbol('id')

No busques **int** que no está. Number se representa siempre en coma flotante de doble precisión (el estándar IEEE 754), tanto si el valor tiene decimales como si no. Es una decisión de diseño del lenguaje: simplifica el sistema de tipos a cambio de ceder algo de precisión y control fino que sí tienen lenguajes con tipos numéricos separados.

```
const edad = 34; // number
const nombre = 'Carlos'; // string
const activo = true; // boolean
let direccionEnvio; // undefined (declarada, sin valor)
const telefonoSecundario = null; // null (ausencia intencionada de valor)
```

Importante: undefined y null no son lo mismo, aunque ambos representan "ausencia de valor". undefined significa que la variable no ha sido inicializada (o que una propiedad o función no existe). null significa que alguien, deliberadamente, ha indicado que no hay valor. Como norma profesional: deja que undefined ocurra de forma natural (no lo asignes a mano) y usa null cuando quieras expresar explícitamente "aquí no hay nada" (por ejemplo, un campo fechaBaja: null en un usuario que sigue activo).

string: comillas simples, dobles y plantillas literales

En JavaScript existen tres formas de delimitar una cadena de texto: comillas simples ('...'), comillas dobles ("...") y comillas invertidas o *backticks* (`...`). Las dos primeras son funcionalmente idénticas: no hay ninguna diferencia de comportamiento entre usar una u otra, solo una cuestión de estilo. Lo importante es elegir una y mantenerla de forma consistente en todo el proyecto.

```
const simples = 'Producto añadido';
const dobles = "Producto añadido";
// Ambas líneas son equivalentes: JavaScript no distingue entre ellas
```

Nota: En desarrollo profesional es habitual fijar una única convención (normalmente comillas simples) mediante un formateador como Prettier, para que todo el equipo escriba el código de la misma forma sin tener que pensarlo.

También se usan cuando necesitamos poner, por ejemplo, comillas dobles dentro de una cadena con comillas simples.

```
let cadena = "Esto es 'importante'"
```

Concatenación

Concatenar es unir dos o más cadenas de texto en una sola, usando el operador +. Aquí veremos el problema que más quebraderos de cabeza da en código real.

```
const nombre = 'Ana';
const apellido = 'García';

console.log(nombre + ' ' + apellido); // 'Ana García'
```

Hasta aquí, todo previsible: dos strings con + dan un string más largo.

El problema: + con un número y un string no suma, concatena

El operador + tiene una regla poco intuitiva: si al menos uno de los dos operandos es un string, el otro se convierte a string, y se concatenan. No se suma nunca en ese caso.

```
console.log('5' + 3); // '53' (no 8)
console.log(3 + '5'); // '35' (no 8)
console.log('El total es ' + 10 + 5); // 'El total es 105' (no 'El total es 15')
```

Esa última línea es la que más sorprende. JavaScript evalúa + de izquierda a derecha:

- 'El total es ' + 10 → como hay un string, concatena → 'El total es 10'
- 'El total es 10' + 5 → sigue habiendo un string, concatena otra vez → 'El total es 105'

El 5 nunca llegó a sumarse a nada; se pegó como texto al final.

¿Qué resultado da la siguiente línea?

```
console.log(2 + 8 + '5');
```

Dónde aparece este bug en la práctica

El caso más habitual, y el que probablemente más te vas a encontrar, es con datos que vienen de prompt() o de un formulario: **siempre llegan como string**, aunque parezcan un número.

```
const cantidad1 = prompt('¿Cuántas unidades del producto A?'); // el usuario escribe "3"
const cantidad2 = prompt('¿Cuántas unidades del producto B?'); // el usuario escribe "5"

console.log(cantidad1 + cantidad2); // '35', no 8
```

Aunque el usuario ha escrito números, prompt() los devuelve como texto, así que + los concatena en vez de sumarlos. Este es, probablemente, el ejemplo más realista de por qué hay que tener el problema en la cabeza.

El caso contrario: querías concatenar y te encuentras una resta

El problema también funciona al revés, y ahí -, * y / se comportan de forma totalmente distinta a +: fuerzan siempre la conversión a número, nunca concatenan.

```
console.log('10' - '4'); // 6 (número)
console.log('10' + '4'); // '104' (string, con el mismo tipo de operandos)
```

Es decir: + es el único operador aritmético con este doble comportamiento (suma o concatena, según el tipo). El resto de operadores matemáticos siempre convierten a número, sin excepción.

La forma profesional de evitar el problema: plantillas literales

Cuando lo que se busca es construir un mensaje con números incrustados (no operar con ellos), mezclar + con strings y números es exactamente donde se cuela el bug con más frecuencia, porque es fácil olvidar un paréntesis o el orden de las sumas. Las plantillas literales evitan el problema de raíz, porque cada expresión dentro de \${ } se evalúa por separado, sin encadenar sumas.

¿Para qué sirven las plantillas literales?

- **Interpolación de expresiones:** permiten insertar el valor de una variable o el resultado de una expresión directamente dentro del texto, sin concatenar con +. La sintaxis es \${ expresión }: todo lo que se escriba dentro de las llaves se evalúa como código JavaScript y el resultado se inserta en ese punto de la cadena.
- **Cadenas multilínea:** permiten que el texto ocupe varias líneas tal cual se escribe, sin unir varias cadenas con + ni usar el carácter especial de salto de línea.

Con comillas simples, mostrar un mensaje con dos variables obliga a concatenar así:

```
const producto = 'ratón inalámbrico';
const precio = 29.99;

const mensajeConcatenado = 'El producto ' + producto + ' cuesta ' +
precio.toFixed(2) + ' €';
```

Con una plantilla literal, la misma línea queda así de legible:

```
const mensaje = `El producto "${producto}" cuesta ${precio.toFixed(2)} €`;
console.log(mensaje); // El producto "ratón inalámbrico" cuesta 29.99 €
```

Y para el caso de cadenas multilínea, esto ya no es posible con comillas simples o dobles (darían un error de sintaxis), pero sí con plantilla literal:

```
const plantillaEmail = `Estimado cliente,

Su pedido ha sido confirmado.
Gracias por su compra.`;
```

Solucionando el caso de la suma que al final concatena:

```
const cantidad1 = 3;
const cantidad2 = 5;

// Con +: hay que tener cuidado con el orden y los paréntesis
console.log('Total: ' + (cantidad1 + cantidad2)); // 'Total: 8' - funciona,
pero el paréntesis es fácil de olvidar
```

```
console.log('Total: ' + cantidad1 + cantidad2); // 'Total: 35' - bug clásico, sin el paréntesis

// Con plantilla literal: la suma ocurre dentro de ${ }, aislada, sin ambigüedad
console.log(`Total: ${cantidad1 + cantidad2}`); // 'Total: 8', siempre correcto
```

Importante: Las plantillas literales se delimitan siempre con comillas invertidas (la tecla situada normalmente junto a P, no con un apóstrofe). Si se usan comillas simples o dobles, `${producto}` no se interpola: aparece tal cual, como texto literal.

Nota: En desarrollo profesional se prefieren las plantillas literales (comillas invertidas) frente a la concatenación con `+`, porque el resultado es mucho más legible cuando hay varias variables involucradas.

En cuanto a los strings, podemos usar caracteres especiales utilizando el carácter de escape `"\"`:

<code>\n</code>	Una nueva línea
<code>\t</code>	Tabulador
<code>\'</code>	Comilla simple
<code>\"</code>	Comilla doble
<code>\\</code>	Barra inclinada

El operador `typeof`

Permite comprobar el tipo de un valor en tiempo de ejecución.

```
console.log(typeof 42); // 'number'
console.log(typeof 'Hola'); // 'string'
console.log(typeof true); // 'boolean'
console.log(typeof undefined); // 'undefined'
console.log(typeof null); // 'object' (ver nota siguiente)
console.log(typeof {}); // 'object'
console.log(typeof []); // 'object'
console.log(typeof function(){}); // 'function'
```

Importante: `typeof null` devuelve `'object'`. Es un error histórico del propio lenguaje (arrastrado desde su primera versión, en 1995, por motivos de compatibilidad) y no se puede corregir sin romper páginas web existentes. Para comprobar si un valor es específicamente `null`, se compara de forma directa: `valor === null`.

1.9 Funciones para cadenas de texto

A veces necesitamos usar el método `"toString()"` para poder aplicar estos métodos.

`.length`

Número de caracteres de la cadena (es una propiedad, no un método: sin paréntesis).

```
const cad = 'hola';
console.log(cad.length); // 4
```

Concatenación: `+` y `.concat()`

```
const mensaje1 = 'Hola';
```

```
const mensaje2 = mensaje1.concat(' Mundo'); // 'Hola Mundo'
```

Nota: .concat() funciona, pero en código moderno se usa casi siempre + o, mejor aún, una plantilla literal. concat() se enseña sobre todo para reconocerlo si aparece en código de terceros.

.toUpperCase() / .toLowerCase()

Transforma todos los caracteres en mayúsculas/minúsculas

```
const mensaje = 'Hola';  
console.log(mensaje.toUpperCase()); // 'HOLA'  
console.log(mensaje.toLowerCase()); // 'hola'
```

.charAt(posicion)

Devuelve el carácter en esa posición. Si no existe, devuelve "" (cadena vacía), no undefined.

```
const mensaje = 'Hola';  
console.log(mensaje.charAt(0)); // 'H'  
console.log(mensaje.charAt(2)); // 'l'  
console.log(mensaje.charAt(9)); // '' (fuera de rango)
```

Nota: una cadena se puede tratar como un array de caracteres, así que mensaje[2] es equivalente a mensaje.charAt(2). La diferencia es que mensaje[9] (fuera de rango) devuelve undefined, mientras que charAt(9) devuelve "".

.indexOf() / .lastIndexOf()

.indexOf() busca la **primera** coincidencia; .lastIndexOf(), la **última**. Ambas devuelven -1 si no encuentran nada, y funcionan igual con un carácter suelto o con una palabra completa. También funciona en Arrays.

```
const mensaje = 'Hola caracola';  
console.log(mensaje.indexOf('a')); // 3 (primera 'a')  
console.log(mensaje.lastIndexOf('a')); // 12 (última 'a')  
console.log(mensaje.indexOf('b')); // -1  
  
const frase = 'Hola que tal';  
console.log(frase.indexOf('que')); // 5
```

Ejercicios · Cadenas

Ejercicio1. Crear una función que pida una cadena por teclado y muestre su tamaño.

Ejercicio2. Crea una función que pida la clave Secreta por pantalla, si la introduce correctamente aparece el mensaje “Has acertado” (también serán válidas: secreta, SECRETA, SeCrEtA...)

.substring(inicio, fin)

Extrae una porción de texto. El segundo parámetro es opcional (si se omite, corta hasta el final).

```
const mensaje = 'Hola Mundo';  
console.log(mensaje.substring(2)); // 'la Mundo'  
console.log(mensaje.substring(1, 8)); // 'ola Mun' (el índice final no se incluye)
```

Importante: .substring() tiene dos comportamientos que sorprenden si no se conocen:

- Un valor negativo se trata como 0: `mensaje.substring(-2)` da 'Hola Mundo' completo, no los dos últimos caracteres (para eso está `.slice()`, que sí admite negativos).
- Si fin es menor que inicio, JavaScript los intercambia automáticamente: `mensaje.substring(5, 0)` da el mismo resultado que `mensaje.substring(0, 5)`.

.substr(inicio, numCaracteres)

Corta desde inicio, un número de caracteres concreto (en vez de hasta un índice final).

```
console.log('JavaScript'.substr(4, 6)); // 'Script'
console.log('Finland'.substr(3, 4));    // 'land'
```

Importante: `.substr()` está marcado como *legacy* (obsoleto) en la especificación del lenguaje, aunque todos los navegadores lo siguen soportando por compatibilidad. En código profesional se prefiere `.slice(inicio, fin)` (que admite índices negativos y no está en desuso) para conseguir el mismo resultado.

.split(separador)

Convierte una cadena en un array, cortando por el separador indicado. Es la operación inversa de `.join()` (que verás en la parte de arrays).

```
const mensaje = 'Hola Mundo, soy una cadena';
console.log(mensaje.split(' ')); // ['Hola', 'Mundo,', 'soy', 'una', 'cadena']

console.log('JavaScript'.split('')); //
['J','a','v','a','S','c','r','i','p','t'] (separador vacío: cada carácter)
```

.trim()

Elimina espacios en blanco al principio y al final

```
const nombreProducto = ' Teclado Mecánico RGB ';
console.log(nombreProducto.trim()); // 'Teclado Mecánico RGB'
```

Ejercicios · Cadenas

Ejercicio 3. Función que defina una variable cadena con "hola que tal yo bien" y busque "tal" para mostrar la cadena desde "tal" al final.

.replace() / .replaceAll()

Sustituye una cadena de texto por otra. También admite expresiones regulares (que ya se verán más adelante). `replace` reemplaza la primera coincidencia y `replaceAll` todas.

```
const str = 'El script es útil. El script es interesante.';

console.log(str.replace('script', 'código')); // 'El código es útil. El
script es interesante.' (solo la 1ª)
console.log(str.replaceAll('script', 'código')); // 'El código es útil. El
código es interesante.' (todas)
```

.padStart(longitud, relleno)

Rellena la cadena actual con una cadena dada (esta cadena puede repetirse varias veces), de modo que la cadena resultante alcance una longitud dada. El relleno es aplicado desde el inicio (izquierda) de la cadena actual

```
function generarReferencia(numeroPedido) {
  return `PED-${String(numeroPedido).padStart(6, '0')}`;
}
```

```
console.log(generarReferencia(42)); // 'PED-000042'
'abc'.padStart(10, "R"); // "RRRRRRRabc" (rellena con un carácter hasta
llegar a 10)
'abc'.padStart(10); // "      abc" (si no se pasa el segundo parámetro, usa
el espacio en blanco)
'abc'.padStart(10, "foo"); // "foofoofabc" (también se puede rellenar con más
de un carácter)
'1'.padStart(2, "0"); // "01" (este ejemplo es útil para mostrar segundos,
minutos, ...)
```

slice, corta un rango de elementos de la cadena. Tiene 2 parámetros, la posición inicial y la posición final(no incluida). Si no se incluye la posición inicial, corta hasta el final de la cadena.

repeat, hay una función que permite repetir una cadena un número de veces, es útil si queremos mostrar por ejemplo, tres `<td></td>` y en vez de hacer un for entre 1 y 3, hacemos un repeat:

```
let resultado = "<td></td>".repeat(3)
//Resultado = <td></td><td></td><td></td>
```

Ejercicios · Cadenas

Ejercicio 4. Crear una función que tenga como parámetro de entrada un DNI y le ponga un 0 o no al principio según corresponda (si tiene 7 caracteres, hay que ponerle un 0 delante).

Ejercicio 5. Crea una función que muestre por pantalla cuántas palabras tiene una frase, mostrando: “Esta frase tiene X palabras”.

1.10 Funciones para los números

toFixed(dígitos): devuelve el número original con tantos decimales como los indicados por el parámetro dígitos y realiza los redondeos necesarios.

```
let numero1 = 4564.34567;
numero1.toFixed(2); // 4564.35
numero1.toFixed(6); // 4564.345670
numero1.toFixed(); // 4564
```

El objeto Math

Math es distinto de String, Number o Date: no es un constructor (nunca se escribe new Math()), sino un único objeto con métodos y constantes matemáticas listos para usar.

Método / propiedad	Qué hace
Math.round(x)	Redondea al entero más cercano
Math.ceil(x)	Redondea siempre hacia arriba
Math.floor(x)	Redondea siempre hacia abajo

Método / propiedad	Qué hace
Math.trunc(x)	Elimina la parte decimal, sin redondear
Math.abs(x)	Valor absoluto
Math.pow(base, exp)	Potencia (equivalente a base ** exp)
Math.sqrt(x)	Raíz cuadrada
Math.max(...) / Math.min(...)	Máximo o mínimo de varios valores
Math.random()	Número decimal aleatorio entre 0 (incluido) y 1 (excluido)

```
console.log(Math.round(4.5)); // 5
console.log(Math.ceil(4.1)); // 5
console.log(Math.floor(4.9)); // 4
console.log(Math.max(12, 45, 3, 8)); // 45
```

Importante: Math.round() y toFixed() (visto en el apartado anterior) no son intercambiables. Math.round(precio * 100) / 100 devuelve un number redondeado a 2 decimales, listo para seguir operando con él.
precio.toFixed(2) devuelve un string formateado, listo para mostrarlo. Usa cada uno según lo que necesites a continuación: cálculo o presentación.

Generar un número entero aleatorio en un rango

Math.random() por sí solo solo da un decimal entre 0 y 1, que rara vez es útil directamente. La fórmula habitual para obtener un entero aleatorio entre dos valores (ambos incluidos) es:

```
/**
 * Genera un número entero aleatorio entre min y max, ambos incluidos.
 * @param {number} min - Valor mínimo posible.
 * @param {number} max - Valor máximo posible.
 * @returns {number} Entero aleatorio en el rango [min, max].
 */
function numeroAleatorioEntre(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

console.log(numeroAleatorioEntre(1, 6)); // simula una tirada de dado
```

1.11 Conversión de tipos de datos

JavaScript realiza conversiones de tipo de dos formas: de manera **implícita** (el propio lenguaje decide convertir un valor sin que se le pida) y de manera **explícita** (el desarrollador convierte un valor de un tipo a otro de forma deliberada).

Conversión implícita (*coerción*)

Ocurre automáticamente al operar con valores de tipos distintos. Es una de las zonas del lenguaje que genera más comportamientos inesperados si no se conoce bien.

```
console.log('5' + 3); // '53' (el + con un string concatena, no suma)
console.log('5' - 3); // 2 (el - fuerza conversión a number)
console.log('5' * '2'); // 10 (* también fuerza conversión a number)
console.log(5 + true); // 6 (true se convierte a 1)
console.log(5 + false); // 5 (false se convierte a 0)
console.log('5' == 5); // true (== convierte antes de comparar)
console.log('5' === 5); // false (=== NO convierte, compara tipo y valor)
```

Importante: El operador == (doble igual) aplica reglas de conversión que a veces producen resultados poco intuitivos (por ejemplo, " == 0 da true, y null == undefined también da true). El operador === compara sin convertir, por lo que el resultado es siempre predecible.

Conversión explícita

El desarrollador indica de forma clara a qué tipo quiere convertir un valor, usando funciones integradas del lenguaje.

parseInt(cadena,base): Recibe una cadena y una base (opcional). Devuelve un valor numérico resultante de convertir la cadena en un número en la base indicada. Ejemplo: parseInt("0xF", 16) devuelve un 15. Si no se le pasa una base, transforma una cadena en un número.

parseFloat(cadena): Convierte la cadena en un número y lo devuelve.

```
// A number
parseInt('42px'); // 42 (parseInt lee hasta donde puede)
parseFloat('3.14 €'); // 3.14

// A string
String(42); // '42'
(42).toString(); // '42'

// A boolean
Boolean(0); // false
Boolean(''); // false
Boolean('0'); // true (una cadena no vacía siempre es truthy)
```

Valores *truthy* y *falsy*

Cuando JavaScript necesita interpretar un valor como booleano (por ejemplo, en la condición de un if), aplica una conversión implícita a true o false. Solo existen ocho valores *falsy*; todo lo demás es *truthy*.

Valores falsy (se convierten a false)	Ejemplos de valores truthy
false, 0, -0, 0n, "", null, undefined, NaN	'0', 'false', '', [], {}, cualquier número distinto de 0

Truco para recordarlo: **falsy = inexistente, cero, falso o error numérico.**

```
const carrito = [];

if (carrito) {
  console.log('Este bloque SÍ se ejecuta'); // un array vacío es truthy
}

if (carrito.length) {
  console.log('Este bloque NO se ejecuta'); // carrito.length es 0, que es falsy
}
```

Nota: Este último ejemplo es un patrón muy habitual en desarrollo profesional: en lugar de escribir `if (carrito.length > 0)`, muchos equipos escriben directamente `if (carrito.length)`, apoyándose en que 0 es *falsy*. Es válido, pero conviene usarlo con cuidado: si `carrito.length` pudiera no existir (por ejemplo, si `carrito` fuera `undefined`), el código lanzaría un error antes de llegar a evaluar la condición.

```
const nombre = '';

if (nombre) {
  console.log('Hay nombre');
} else {
  console.log('No hay nombre');
}
```

1.12 Operadores

Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	$5 + 2 = 7$
-	Resta	$5 - 2 = 3$
*	Multiplicación	$5 * 2 = 10$
/	División	$5 / 2 = 2.5$
%	Resto(modulo)	$5 \% 2 = 1$
**	Potencia	$5 ** 2 = 25$
++ / --	Incremento / decrement	let x = 5; x++; = 6

```
// El operador % es muy habitual para comprobar paridad
function esPar(numero) {
  return numero % 2 === 0;
}
console.log(esPar(4)); // true
console.log(esPar(7)); // false
```

Operadores de asignación

Operador	Equivalente a
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>

<code>x ??= y</code>	<code>x = x ?? y</code> (asigna solo si x es null o undefined=)
----------------------	---

Operadores de incremento y decremento

Operador	Equivalente a
<code>var1++</code>	<code>Var1 = var1 + 1</code>
<code>var1--</code>	<code>Var1 = var1 - 1</code>

Operadores de comparación

Operador	Equivalente a
<code>=== / !==</code>	Igual / distinto (sin conversión de tipo; siempre recomendados)
<code>== / !=</code>	Igual / distinto (con conversión de tipo; evitar)
<code>> / <</code>	Mayor que / menor que
<code>>= / <=</code>	Mayor o igual que / menor o igual que

Operadores lógicos

Operador	Descripción	Ejemplo
<code>&&</code>	Y lógico	<code>edad >=18 && tieneDni</code>
<code> </code>	O lógico	<code>esAdmin esEditor</code>
<code>!</code>	Negación	<code>!estaActivo</code>
<code>??</code>	Coalescencia nula (usa el valor de la derecha solo si el de la izquierda es null o undefined)	<code>Nombre ?? 'Anónimo'</code>
<code>?.</code>	Encadenamiento opcional (accede a una propiedad solo si el objeto existe)	<code>usuario?.direccion?.ciudad</code>

```
const descuentoAplicado = 0;

// || trata 0 como falsy, así que aquí NO se comporta como se espera
const descuentoMostrado1 = descuentoAplicado || 'Sin descuento';
console.log(descuentoMostrado1); // 'Sin descuento' (incorrecto: 0 SÍ es un
descuento válido)

// ?? solo entra en juego si el valor es null o undefined
const descuentoMostrado2 = descuentoAplicado ?? 'Sin descuento';
console.log(descuentoMostrado2); // 0 (correcto: respeta el valor 0)
```

Importante: Este es uno de los errores más frecuentes en código profesional: usar `||` para poner un valor por defecto cuando en realidad se necesita `??`. `||` sustituye cualquier valor *falsy* (0, "", false), mientras que `??` solo sustituye null o undefined. Si un 0, una cadena vacía o un false son valores legítimos en tu aplicación, usa siempre `??`.

1.13 Comentarios al código

Los comentarios son texto que el motor de JavaScript ignora al ejecutar el programa, pero que resulta fundamental para que otras personas (o el propio autor, meses después) entiendan el código.

Comentarios de una línea

```
// Calcula el precio final aplicando el IVA vigente
const precioConIva = precioBase * 1.21;
```

Comentarios multilínea

```
/*
  Este bloque calcula el precio final de un pedido,
  incluyendo IVA y gastos de envío según la zona.
*/
const precioFinal = calcularPrecioFinal(pedido);
```

Comentarios de documentación (JSDoc)

En proyectos profesionales, las funciones se documentan con un formato estandarizado llamado **JSDoc**, que además permite que el editor de código (VS Code) muestre autocompletado y ayuda contextual al usar esa función desde cualquier otro fichero. Una vez creada la función, escribe encima **/**** y pulsa **INTRO**, VSCode te generará parte del código, solo tienes que rellenar el tipo y la descripción.

```
/**
 * Calcula el precio final de un producto aplicando el IVA.
 * @param {number} precioBase - Precio sin impuestos.
 * @param {number} [tipoIva=0.21] - Tipo de IVA a aplicar (por defecto, 21%).
 * @returns {number} Precio final con IVA incluido.
 */
function calcularPrecioConIva(precioBase, tipoIva = 0.21) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}
```

Importante: Un comentario que explica qué hace una línea obvia (`// suma uno a contador sobre contador++;`) no aporta valor y ensucia el código. Un buen comentario explica el por qué de una decisión que no es evidente con solo leer el código (por ejemplo, por qué se aplica un descuento especial en una condición concreta). El mejor comentario, en muchos casos, es un nombre de variable o de función bien elegido que hace innecesario el propio comentario.

1.14 Depurando

La mejor forma de depurar es ejecutando el código paso a paso. Esa funcionalidad nos la dan los navegadores (al menos casi todos).

Al pulsar **F12** en el navegador se nos abre un conjunto de herramientas, entre ellas un depurador. Otra forma de abrir estas herramientas es pulsando con el botón derecho en el navegador y elegir la opción “Inspeccionar”.

Actividades - Operadores, Prompt, ...

Actividad 1. Si nos fijamos en estas líneas de código

```
num1 = 5;
resultado = num1 = 3; // ¿Cuánto vale aquí 'resultado'?
resultado = num1 == 3; // ¿Cuánto vale aquí 'resultado'?
```

Nota: Vamos a aprovechar esta actividad para aprender a utilizar el depurador del navegador y ‘observar’ una variable.

Actividad 2. Escribir el código necesario para que realice esta operación “ $v1 * v2 + v3$ ” y muestre el resultado por consola. Siendo los valores de $v1$, $v2$ y $v3$:

$v1=2$ $v2=4$ $v3=5$

A continuación, hacer lo mismo, pero con esta operación “ $v1 + v2 * v3$ ”. Antes de ejecutarlo, pensad qué resultado obtendremos. ¿Has acertado?

¿Qué orden sigue JS para realizar las operaciones? ¿Cómo podemos cambiarlo?

Actividad 3. ¿Cuál sería el resultado de las siguientes operaciones? $a=5$; $a/=2$; $a+=1$; $a*=3$; $a--$; $a++$;

Actividad 4. Suma tres números tecleados por usuario y calcula la media, mostrando en la consola:

“La suma es: RESULTADO_SUMA”

“La media es: MEDIA”

Actividad 5. Pide al usuario una cantidad de "millas náuticas" y muestra la equivalencia en metros.

Actividad 6. Dado el siguiente código:

```
let a = 5;
```

```
let b = a++;
```

Y este otro:

```
let a = 5;
```

```
let b = ++a;
```

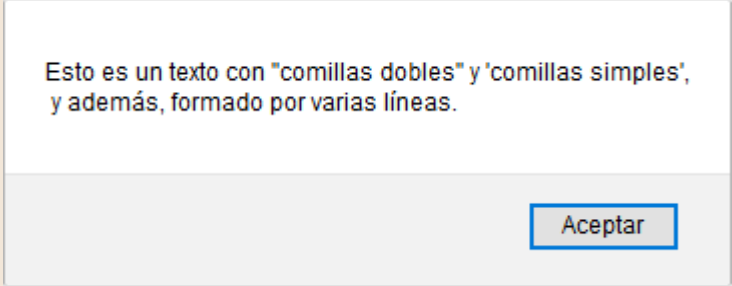
¿Tendrá la variable ‘b’ el mismo valor en ambos scripts? ¿Por qué?

Actividad 7. ¿Que mostrará el siguiente código en la consola? ¿Por qué?

```
let foo = 'hola';
```

```
console.log( parseInt(foo) );
```

Actividad 8. Mostrar un mensaje con alert, como el que vemos en la imagen:



Esto es un texto con "comillas dobles" y 'comillas simples',
y además, formado por varias líneas.

Aceptar

2. Estructuras de control

Hasta ahora el código se ha ejecutado siempre de arriba abajo, línea a línea. Las estructuras de control permiten decidir qué código se ejecuta (condicionales) y repetir código varias veces (bucles).

2.1 Sentencias condicionales: if, else if, else

Una sentencia condicional ejecuta un bloque de código solo si se cumple una condición. La condición es cualquier expresión que JavaScript pueda interpretar como truthy o falsy.

if simple

```
const totalCarrito = 45;  
  
if (totalCarrito > 50) {  
  console.log('Envío gratuito');  
}
```

```
}
```

if / else

Añade un bloque alternativo que se ejecuta cuando la condición no se cumple.

```
const totalCarrito = 45;

if (totalCarrito > 50) {
  console.log('Envío gratuito');
} else {
  console.log('Gastos de envío: 4.95 €');
}
```

if / else if / else

Permite encadenar varias condiciones que se evalúan en orden. En cuanto una se cumple, se ejecuta su bloque y se ignoran las siguientes.

```
function calcularGastosEnvio(totalCarrito) {
  if (totalCarrito >= 50) {
    return 0;
  } else if (totalCarrito >= 30) {
    return 2.95;
  } else {
    return 4.95;
  }
}

console.log(calcularGastosEnvio(60)); // 0
console.log(calcularGastosEnvio(35)); // 2.95
console.log(calcularGastosEnvio(10)); // 4.95
```

Importante: Escribe siempre las llaves { }, incluso cuando el bloque tenga una sola sentencia. JavaScript permite omitirlas para bloques de una línea, pero esto es una fuente clásica de errores: si en el futuro se añade una segunda línea al bloque sin añadir las llaves, esa segunda línea queda fuera del if sin que salte ningún error, y el programa falla de forma silenciosa. Los linters profesionales suelen obligar a poner siempre las llaves.

if anidados

Un if puede contener otro if dentro de su bloque, para comprobar condiciones adicionales solo cuando la primera se cumple.

```
function puedeComprarAlcohol(edad, tieneIdentificacion) {
  if (edad >= 18) {
    if (tieneIdentificacion) {
      return 'Venta permitida';
    } else {
      return 'Debe presentar identificación';
    }
  }

  return 'Venta no permitida';
}
```

Nota: El ejemplo anterior también podría resolverse con una sola condición usando el operador `&&`: `if (edad >= 18 && tieneIdentificacion)`. Anidar es útil cuando cada nivel necesita su propia rama `else` con un mensaje distinto, como en este caso; si solo hay un resultado final, combinar condiciones con operadores lógicos suele ser más legible que anidar.

El operador ternario: una alternativa compacta al `if / else`

Cuando un `if / else` solo sirve para decidir el valor que se asigna a una variable, existe una forma más compacta de escribirlo: el **operador ternario**. Se llama así porque es el único operador de JavaScript que trabaja con tres operandos: una condición, un valor si es verdadera y un valor si es falsa.

```
// Con if / else: cuatro líneas para asignar un único valor
let tipoEntrada;
if (edad >= 18) {
  tipoEntrada = 'Adulto';
} else {
  tipoEntrada = 'Menor';
}

// Con operador ternario: la misma decisión, en una línea
const edad = 16;
const tipoEntradaTernario = edad >= 18 ? 'Adulto' : 'Menor';
console.log(tipoEntradaTernario); // 'Menor'
```

La sintaxis es `condición ? valorSiVerdadero : valorSiFalso`. Al ser una expresión (devuelve un valor, no solo ejecuta código), se puede usar directamente dentro de otra expresión, como una plantilla literal:

```
const stock = 0;
console.log(`Estado: ${stock > 0 ? 'Disponible' : 'Agotado'}`);
// Estado: Agotado
```

Nota: El operador ternario es adecuado para decisiones simples de una sola línea que producen un valor. Si la lógica requiere ejecutar varias sentencias, o hay más de dos caminos posibles, un `if / else if / else` normal es más legible. Encadenar varios ternarios seguidos (`a ? b : c ? d : e`) es una práctica que la mayoría de guías de estilo profesionales desaconsejan por su baja legibilidad.

2.2 La sentencia `switch`

Cuando una misma variable se compara contra muchos valores posibles, encadenar `else if` se vuelve repetitivo. La sentencia `switch` resuelve ese caso concreto: compara una expresión contra una lista de valores (`case`) y ejecuta el bloque correspondiente al primero que coincida.

```
function descripcionEstadoPedido(estado) {
  switch (estado) {
    case 'pendiente':
      return 'El pedido está pendiente de confirmación';
    case 'enviado':
      return 'El pedido ya ha salido del almacén';
    case 'entregado':
```

```
    return 'El pedido ha sido entregado';
  default:
    return 'Estado desconocido';
  }
}
console.log(descripcionEstadoPedido('enviado')); // 'El pedido ya ha salido
del almacén'
```

default es opcional, y se ejecuta cuando ningún case coincide con el valor evaluado. Es equivalente al else final de una cadena de **if / else if**.

También se puede utilizar el switch utilizando un rango de valores en el “case” en vez de un único valor:

```
let num = 5
switch (true) {
  case num > 0:
    ...
    break;
  case (num >= 5 && num < 10):
    ...
    break;
  default:
    ...
    break;
}
```

El problema del fall-through

Cuando un bloque case no termina con return o break, la ejecución continúa cayendo al siguiente case, aunque su valor ya no coincida con la expresión evaluada. Este comportamiento se llama fall-through y es una de las fuentes de errores más comunes con switch.

```
function descripcionEstadoConError(estado) {
  let mensaje;

  switch (estado) {
    case 'pendiente':
      mensaje = 'Pendiente'; // falta el break
    case 'enviado':
      mensaje = 'Enviado';
      break;
    default:
      mensaje = 'Desconocido';
  }

  return mensaje;
}

console.log(descripcionEstadoConError('pendiente')); // 'Enviado' (¡no era lo
esperado!)
```


2.3 Bucles: while y do...while

Un bucle repite un bloque de código mientras se cumpla una condición. while y do...while son los dos bucles más básicos: la diferencia está en cuándo se comprueba la condición.

while

Comprueba la condición **antes** de cada repetición. Si la condición es falsa desde el principio, el bloque no se ejecuta ni una sola vez.

```
let entradasDisponibles = 3;

while (entradasDisponibles > 0) {
  console.log(`Quedan ${entradasDisponibles} entradas`);
  entradasDisponibles--;
}

console.log('Entradas agotadas');
```

Importante: Toda variable que aparezca en la condición del bucle debe modificarse dentro de su bloque (en el ejemplo, entradasDisponibles--). Olvidar ese paso es la causa más frecuente de un bucle infinito: la condición nunca deja de cumplirse y el programa se queda bloqueado (en un navegador, la pestaña deja de responder).

do...while

Comprueba la condición **después** de cada repetición, lo que garantiza que el bloque se ejecute siempre al menos una vez, incluso si la condición ya es falsa desde el principio. Es útil, por ejemplo, para pedir un dato al usuario hasta que sea válido.

```
let edadIntroducida;

do {
  edadIntroducida = parseInt(prompt('Introduce tu edad (0-120):'));
} while (edadIntroducida < 0 || edadIntroducida > 120)

console.log(`Edad registrada: ${edadIntroducida}`);
```

Nota: Recuerda que prompt siempre devuelve un string (o null si se cancela), por lo que hay que convertirlo con parseInt antes de compararlo numéricamente, como se explicó en el apartado de conversión de tipos.

2.4 El bucle for

Cuando se conoce de antemano cuántas veces hay que repetir un bloque, for es la opción más habitual: reúne en una sola línea la inicialización, la condición y la actualización del contador, en lugar de repartirlas como en un while.

```
for (let i = 1; i <= 5; i++) {
  console.log(`Línea de factura número ${i}`);
}
```

Las tres partes del paréntesis se ejecutan en un orden concreto:

Parte	Cuando se ejecuta	En el ejemplo
Inicialización	Una sola vez, al empezar	let i = 1
Condición	Antes de cada vuelta	i <= 5
Actualización	Después de cada vuelta	i++

También puede recorrer en sentido descendente, o avanzar de dos en dos, cambiando la actualización:

```
// Cuenta atrás
for (let i = 5; i >= 1; i--) {
  console.log(i);
}

// Solo números pares
for (let i = 0; i <= 10; i += 2) {
  console.log(i);
}
```

Nota: Como norma general: usa for cuando el número de repeticiones se conoce de antemano (recorrer un rango, repetir algo N veces), y while cuando depende de una condición externa que no se puede calcular por adelantado (esperar a que el usuario introduzca un dato válido, repetir hasta que ocurra algo). Más adelante, se verán además for...of y métodos de array como alternativa específica para recorrer colecciones de datos.

2.5 for...in y for...of

Los dos recorren "algo", pero recorren cosas distintas y están pensados para casos distintos. Aquí tienes ambos explicados uno junto al otro.

for...of: recorre valores de un iterable

Recorre directamente los valores de una colección iterable: arrays, strings, y otras estructuras que se verán más adelante (como los Map y Set, o las colecciones de nodos del DOM).

```
const productos = ['Teclado', 'Ratón', 'Monitor'];

for (const producto of productos) {
  console.log(producto);
}

// 'Teclado'
// 'Ratón'
// 'Monitor'
```

for...in: recorre las claves de un objeto

for...in recorre las propiedades **enumerables** de un objeto, dándote su nombre (la clave), no su valor directamente.

```
const producto = { nombre: 'Teclado', precio: 45, disponible: true };
```

```
for (const clave in producto) {  
  console.log(`${clave}: ${producto[clave]}`);  
}  
// 'nombre: Teclado'  
// 'precio: 45'  
// 'disponible: true'
```

Fíjate en que dentro del bucle hace falta `producto[clave]` (notación de corchetes) para obtener el valor; `for...in` solo te da el nombre de la propiedad.

2.6 Sentencias de ruptura: break y continue

Dentro de un bucle, a veces hace falta salir antes de que la condición se cumpla de forma natural, o saltarse una vuelta concreta sin terminar el bucle entero. Para eso existen `break` y `continue`.

break: termina el bucle por completo

Salte inmediatamente del bucle, sin ejecutar el resto de sus vueltas.

```
// Busca el primer número, hasta 100, divisible entre 7 y entre 4 a la vez  
let numeroEncontrado = null;  
  
for (let i = 1; i <= 100; i++) {  
  if (i % 7 === 0 && i % 4 === 0) {  
    numeroEncontrado = i;  
    break; // ya no hace falta seguir comprobando el resto  
  }  
}  
  
console.log(numeroEncontrado); // 28
```

continue: se salta la vuelta actual

Interrumpe únicamente la iteración actual y pasa directamente a comprobar la condición de la siguiente, sin ejecutar el resto del bloque en esa vuelta.

```
// Suma solo los números impares entre 1 y 10  
let sumaImpares = 0;  
  
for (let i = 1; i <= 10; i++) {  
  if (i % 2 === 0) {  
    continue; // se salta los números pares  
  }  
  sumaImpares += i;  
}  
  
console.log(sumaImpares); // 25
```

Nota: `break` ya se había usado antes, dentro de un `switch`, con la misma idea: interrumpir la ejecución en ese punto. En un bucle, además, evita seguir comprobando condiciones que ya no aportan nada, lo cual mejora el rendimiento cuando el bucle recorre muchos elementos.

Actividades – Estructuras de control

De aquí en adelante, cosas que hay que tener en cuenta a la hora de realizar los ejercicios:

- Cuidado con lo que devuelve el prompt => es un string
- Si no funciona el código => mirar la consola (F12) para ver si hay errores de sintaxis.
- Si no funciona el código y no hay errores de sintaxis => depurar (F12)

Actividad 1. Crea un programa que pida al usuario un número y diga si es par.

Actividad 2. Crea un programa que pida al usuario dos números y diga cuál es el mayor de ellos.

Actividad 3. Crea un programa que pida al usuario dos números y diga si el primero es múltiplo del segundo.

Actividad 4. Crea un programa que pida al usuario dos números. Si el segundo no es cero, mostrará el resultado de dividir el primero entre el segundo. Por el contrario, si el segundo número es cero, escribirá "Error: No se puede dividir entre cero".

Actividad 5. Crea un programa que pida al usuario dos números y diga si son iguales o, en caso contrario, cuál es el mayor de ellos.

Actividad 6. Necesitamos un programa que muestre los números del 10 al 20, ambos inclusive. Y a continuación, muestre los números del 20 al 10, es decir, de mayor a menor.

Actividad 7. Crea un programa que escriba en pantalla los números del 1 al 50 que sean múltiplos de 3.

Actividad 8. Crea un programa que muestre los números del 100 al 200 (ambos incluidos) que sean divisibles entre 7 y a la vez entre 3.

Actividad 9. Crea un programa que muestre la tabla de multiplicar del 9:

```
9 x 0 = 0
9 x 1 = 9
9 x 2 = 18
...
9 x 10 = 90
```

Actividad 10. Crea un programa que muestre los primeros ocho números pares: 2 4 6 8 10 12 14 16.

Actividad 11. Crea un programa que muestre los números del 15 al 5, descendiendo.

Actividad 12. Crea un programa que pida al usuario el ancho (por ejemplo, 4) y el alto (por ejemplo, 3) y muestre un rectángulo formado por esa cantidad de asteriscos:

```
****
****
****
```

Actividad 13. Lo mismo que el ejercicio anterior, pero esta vez mostrará una diagonal con otro carácter:

```
O***
*O**
**O*
***O
```

Actividad 14. Crea un programa que pida al usuario su identificador y contraseña de manera indefinida hasta que introduzca el identificador "alibaba" y la contraseña "sesamo". Si al tercer intento no ha introducido los datos correctos, mostrará un mensaje diciendo que el identificador/contraseña no son correctos.

Actividad 15. Crea un "calculador de cuadrados": pedirá al usuario un número y mostrará su cuadrado. Se repetirá mientras el número introducido no sea cero.

Actividad 16. Escribe un código que genere una excepción (por ejemplo, llamar a una función que no exista). Capturar el error y mostrar un mensaje de aviso.

Actividad 17. Crea un programa que contenga un bucle sin fin que muestra el mensaje "Hola" con alert. Cuando llegue a 10 vueltas, se detendrá.

3. Arrays

Un array agrupa varios valores en una sola variable, en un orden determinado. Aquí se estudian a fondo, junto con los métodos que los convierten en una de las herramientas más usadas de todo el lenguaje.

3.1 Crear arrays y acceder a sus elementos

Un array se crea con corchetes, separando sus elementos por comas. Cada elemento tiene una posición numérica, llamada índice, que empieza siempre en 0.

```
const productos = ['Teclado', 'Ratón', 'Monitor'];

console.log(productos[0]); // 'Teclado'
console.log(productos[2]); // 'Monitor'
console.log(productos[5]); // undefined: no existe esa posición
console.log(productos.length); // 3
```

Para acceder al último elemento sin conocer de antemano la longitud del array, el método moderno .at() admite índices negativos, contando desde el final:

```
console.log(productos.at(-1)); // 'Monitor' (el último)
console.log(productos[productos.length - 1]); // 'Monitor' (la forma tradicional, equivalente)
```

Nota: Un array puede contener valores de distintos tipos a la vez ([1, 'dos', true] es válido), pero en código profesional se procura que todos los elementos de un mismo array sean del mismo tipo o tengan la misma forma, precisamente para poder recorrerlos y procesarlos con los métodos que se ven en este tema sin tener que comprobar el tipo de cada elemento uno por uno.

Importante: Evita crear arrays con new Array(). new Array(3) no crea un array con un elemento de valor 3, sino un array vacío de longitud 3 (tres huecos sin definir), un comportamiento que sorprende a quien no lo conoce. La sintaxis de corchetes ([]), usada en todos los ejemplos de este tema, no tiene ninguno de estos casos especiales y es la forma recomendada en cualquier situación.

3.2 Añadir, eliminar y modificar elementos

Método	Qué hace
.push(valor)	Añade un elemento al final; devuelve la nueva longitud
.pop()	Elimina y devuelve el último elemento
.unshift(valor)	Añade un elemento al principio; devuelve la nueva longitud
.shift()	Elimina y devuelve el primer elemento

pop(), elimina el último elemento del array y lo devuelve. El array original se modifica y su longitud disminuye en 1 elemento.

```
let array = [1, 2, 3];
let ultimo = array.pop();
// ahora array = [1, 2], ultimo = 3
```

push(), añade un elemento al final del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
let array = [1, 2, 3];
array.push(4);
// ahora array = [1, 2, 3, 4]
```

shift(), elimina el primer elemento del array y lo devuelve. El array original se ve modificado y su longitud disminuida en 1 elemento.

```
let array = [1, 2, 3];
let primero = array.shift();
// ahora array = [2, 3], primero = 1
```

unshift(), añade un elemento al principio del array. El array original se modifica y aumenta su longitud en 1 elemento. (También es posible añadir más de un elemento a la vez)

```
let array = [1, 2, 3];
array.unshift(0);
// ahora array = [0, 1, 2, 3]
```

Ejemplo completo:

```
const carrito = ['Teclado'];

carrito.push('Ratón');
console.log(carrito); // ['Teclado', 'Ratón']

carrito.unshift('Monitor');
console.log(carrito); // ['Monitor', 'Teclado', 'Ratón']

const ultimoEliminado = carrito.pop();
console.log(ultimoEliminado); // 'Ratón'
console.log(carrito); // ['Monitor', 'Teclado']
```

Modificar un elemento por su índice

```
const productos = ['Teclado', 'Ratón', 'Monitor'];
productos[1] = 'Ratón inalámbrico';
console.log(productos); // ['Teclado', 'Ratón inalámbrico', 'Monitor']
```

Nota: Esto no contradice lo visto sobre const: la variable productos sigue apuntando al mismo array en todo momento, solo se modifica su contenido interno, igual que ocurría al hacer usuario.rol = 'editor' con un objeto declarado con const.

slice, corta un rango de elementos del array y lo devuelve. Tiene 2 parámetros, la posición inicial y la posición final(no incluida). Si no se incluye la posición inicial, corta hasta el final del array.

```
const numbers = [1, 2, 3, 4, 5]

console.log(numbers.slice()) // -> copia todos los elementos
console.log(numbers.slice(0)) // -> copia todos los elementos
console.log(numbers.slice(1, 4)) // -> [2,3,4] // No incluye el índice 4
```

splice, es el método más versátil (y el que más confusión genera) para modificar un array en cualquier punto, no solo en los extremos. Su forma general es `array.splice(inicio, cantidadAEliminar, ...elementosAInsertar)`, y **modifica el array original**. Devuelve un array con los elementos eliminados.

```
const numbers = [1, 2, 3, 4, 5]
console.log(numbers.splice()) // -> elimina todos los ítems

const numbers = [1, 2, 3, 4, 5]
console.log(numbers.splice(0, 1)) // -> elimina el primer elemento

const numbers = [1, 2, 3, 4, 5, 6]
console.log(numbers.splice(3, 3, 7, 8, 9)) // -> [1, 2, 3, 7, 8, 9] //Elimina
los últimos 3 elementos y los reemplaza por 7,8,9.

const pedidos = ['P001', 'P002', 'P003', 'P004'];

// Eliminar: desde el índice 1, elimina 2 elementos
const eliminados = pedidos.splice(1, 2);
console.log(eliminados); // ['P002', 'P003'] (lo eliminado)
console.log(pedidos); // ['P001', 'P004']

// Insertar: desde el índice 1, elimina 0 elementos, e inserta dos nuevos
pedidos.splice(1, 0, 'P005', 'P006');
console.log(pedidos); // ['P001', 'P005', 'P006', 'P004']

// Reemplazar: desde el índice 0, elimina 1 elemento, e inserta uno nuevo en
su lugar
pedidos.splice(0, 1, 'P001-B');
console.log(pedidos); // ['P001-B', 'P005', 'P006', 'P004']
```

Importante: Nunca uses el operador delete para quitar un elemento de un array (`delete pedidos[1]`): deja un hueco (el elemento pasa a ser undefined) sin acortar el array ni reordenar el resto de índices, lo que casi nunca es el resultado que se busca. Para eliminar de verdad un elemento, usa siempre `splice()` (si conoces su posición) o los métodos de transformación que veremos más adelante, como `filter()`.

Crear un array de tamaño fijo: `Array(n)` y `.fill()`

Es una forma rápida de generar un array del tamaño que quieras, todo con el mismo valor inicial.

```
const ochoValoresX = Array(8).fill('X');
console.log(ochoValoresX); // ['X','X','X','X','X','X','X','X']
```

`.concat()`

Se emplea para concatenar los elementos de varios arrays.

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];
const array3 = array1.concat(array2); // [1, 2, 3, 4, 5, 6]
```

[...array1, ...array2] hace lo mismo y es la forma preferida en código moderno.
 .concat() sigue siendo válido y lo verás en código existente.

.join(separador)

Operación inversa de .split(): convierte un array en una cadena de texto. Sin argumento, usa una coma por defecto.

```
const names = ['Ana', 'Luis', 'Marta'];

console.log(names.join());      // 'Ana,Luis,Marta'
console.log(names.join(''));    // 'AnaLuisMarta'
console.log(names.join(' '));   // 'Ana Luis Marta'
console.log(names.join(', '));  // 'Ana, Luis, Marta'
```

Array.isArray()

Comprueba si un valor es realmente un array. Útil porque, como vimos, typeof [] devuelve 'object', no 'array'; esta es la forma correcta de distinguirlo.

```
console.log(Array.isArray([1, 2, 3])); // true
console.log(Array.isArray('hola'));    // false
```

.indexOf() / .lastIndexOf()

.indexOf() busca la **primera** coincidencia; .lastIndexOf(), la **última**. Ambas devuelven -1 si no encuentran nada, y funcionan igual con un carácter suelto o con una palabra completa. Igual que con cadenas.

```
const numbers = [1, 2, 3, 4, 5]
console.log(numbers.indexOf(5)) // -> 4
console.log(numbers.indexOf(0)) // -> -1

const numbers = [1, 2, 3, 4, 5, 3, 1, 2]
console.log(numbers.lastIndexOf(2)) // 7
console.log(numbers.lastIndexOf(0)) // -1
```

.toString()

Convierte un array a texto, uniendo sus elementos con comas (equivale a .join() sin argumentos).

```
const numeros = [1, 2, 3];
console.log(numeros.toString()); // '1,2,3'
```

Ejercicios · Arrays

Ejercicio 1. Crea un array de elementos y muestra dichos elementos por pantalla separados por "#", ahora haz lo mismo con uno bidimensional:

```
let anidado1 = ["anidado1", "anidado2", "anidado3"]
let anidado2 = ["a1", "a2", "a3"]
let matriz = [anidado1, anidado2]
```

Recorre la matriz y muestra los elementos separados por "#", mostrando el mensaje: "Índice X: elem1#elem2#elem3"

Ejercicio 2. Mostrar los elementos de un array del final al principio usando pop.

Ejercicio 3. Nos dan un array de frutas ["banana", "naranja", "mango", "limon"]. Crear un script que pregunte al usuario qué fruta quiere buscar en el array, si no existe dicha

fruta mostrará “Esta fruta no existe en el array” en caso contrario mostrará “Sí hay aguacate en el array”. La fruta se puede poner en mayúsculas y minúsculas y la seguirá encontrando.

reverse(), modifica un array colocando sus elementos en el orden inverso a su posición original:

```
let array = [1, 2, 3];
array.reverse();
// ahora array = [3, 2, 1]
```

sort(), ordena los elementos de un array en orden ascendente.

```
let fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.sort(); // fruits = ["Apple", "Banana", "Mango", "Orange"];
```

Ejercicios · Arrays

Ejercicio 4. Si tenemos una cadena en minúsculas “miel”. ¿Cómo podemos transformarla en una cadena con la primera letra en mayúsculas(Miel)?

Ejercicio 5. Nos dan la lista de la compra en un array(["Leche", "Café", "Té", "Miel"). Crea un botón para añadir “Carne” al principio de la lista de la compra y “Azúcar” al final de la lista. Si ya se ha añadido “Carne” o “Azúcar” no se vuelve a añadir, se muestra el mensaje “Ya se ha añadido este producto a la lista de la compra”.

Crea un botón “Alergias” que pida un producto al que eres alérgico y lo elimina de la lista de la compra, indicando “He eliminado X producto de la cesta porque eres alérgico”, si no se encuentra el producto en la cesta, mostrará “No existen alergias en la cesta”.

Crea un botón “Modificar” que pida un producto a buscar y si lo encuentra modifique su valor, por ejemplo, que busque “Té” y te permita modificarlo a “Té verde”.

3.3 Recorrer un array: for, for...of y forEach

for clásico

Usando el índice, exactamente como se vio anteriormente, pero con la condición ligada a .length.

```
const productos = ['Teclado', 'Ratón', 'Monitor'];

for (let i = 0; i < productos.length; i++) {
  console.log(`${i}: ${productos[i]}`);
}
```

for...of

Ya se ha usado en temas anteriores, con la promesa de explicarlo aquí en detalle. Recorre directamente los **valores** de un array, sin necesidad de manejar un índice.

```
for (const producto of productos) {
  console.log(producto);
}

// 'Teclado'
// 'Ratón'
// 'Monitor'
```

Al no depender de un índice, `for...of` es más legible cuando no hace falta saber en qué posición está cada elemento. Como es una variante de `for`, admite `break` y `continue` con normalidad.

forEach(): recorrer con un método del propio array

`forEach()` ejecuta una función (un *callback*, es decir, una función que se pasa como argumento a otra) una vez por cada elemento del array.

```
productos.forEach((producto, indice) => {
  console.log(`${indice}: ${producto}`);
});
```

El *callback* recibe hasta tres argumentos: el elemento actual, su índice, y el array completo (los dos últimos son opcionales, y se pueden omitir si no se necesitan).

Importante: `forEach()` siempre devuelve `undefined`, y no admite `break` ni `continue`: una vez empieza, recorre el array entero sin posibilidad de interrumpirlo antes de tiempo. Si en algún momento hace falta salir del bucle antes de llegar al final, hay que usar `for` o `for...of` en su lugar.

Ejercicios · Arrays

Ejercicio 6. Crea una función que recorra una cadena y separe los caracteres con un guión (Que no aparezca guión al final).

Ejercicio 7. Función que defina una cadena, la corte y la meta en un array. Luego debe recorrer el array e indicar que en el índice X del array está tal cadena.

3.4 Buscar y comprobar elementos

Método	Qué devuelve
<code>.includes(valor)</code>	<code>true/false</code> según si el array contiene ese valor exacto
<code>.indexOf(valor)</code>	El índice de la primera coincidencia, o -1 si no está
<code>.find(callback)</code>	El primer elemento que cumple la condición, o <code>undefined</code>
<code>.findIndex(callback)</code>	El índice del primer elemento que cumple la condición, o -1
<code>.some(callback)</code>	<code>true</code> si al menos un elemento cumple la condición
<code>.every(callback)</code>	<code>true</code> solo si todos los elementos cumplen la condición

`.includes()` e `.indexOf()` sirven para valores simples y exactos. Cuando los elementos son objetos (como ya se vio brevemente en el Tema 2 y en el Tema 7), hace falta una condición más elaborada, y para eso están `find`, `findIndex`, `some` y `every`, que reciben un `callback` igual que `forEach`.

```
const inventario = [
  { sku: 'TC-01', nombre: 'Teclado', stock: 12 },
```

```
{ sku: 'RT-02', nombre: 'Ratón', stock: 0 },  
{ sku: 'MN-03', nombre: 'Monitor', stock: 5 }  
];  
  
const raton = inventario.find(articulo => articulo.sku === 'RT-02');  
console.log(raton); // { sku: 'RT-02', nombre: 'Ratón', stock: 0 }  
  
const hayAgotados = inventario.some(articulo => articulo.stock === 0);  
console.log(hayAgotados); // true  
  
const todoDisponible = inventario.every(articulo => articulo.stock > 0);  
console.log(todoDisponible); // false
```

Nota: La sintaxis { clave: valor } usada en inventario es un objeto literal, ya adelantado de forma puntual en temas anteriores. Su sintaxis completa, junto con clases y el resto de programación orientada a objetos, se estudia más adelante.

4. Funciones

Las funciones permiten agrupar código para reutilizarlo con distintos datos de entrada, en lugar de repetirlo. Este tema cubre las distintas formas de declararlas, cómo reciben y devuelven datos, y un concepto clave para entender cómo funciona JavaScript por dentro: las clausuras.

4.1 Declaración de funciones: declaration vs.expression

Existen dos formas principales de definir funciones, con una diferencia de comportamiento importante entre ellas.

Function declaration

Se define con la palabra clave function seguida de un nombre, como sentencia independiente.

```
function calcularIva(precio) {  
  return precio * 0.21;  
}
```

Function expression

Se define asignando una función, sin nombre propio, a una variable.

```
const calcularIva = function (precio) {  
  return precio * 0.21;  
};
```

Importante: La diferencia práctica más relevante está en el hoisting. Una function declaration se eleva por completo, con su cuerpo incluido, así que se puede llamar antes de la línea donde aparece escrita en el código. Una function expression se comporta como cualquier variable const o let: existe, pero no se puede usar antes de la línea donde se le asigna la función (queda en la temporal dead zone).

```
console.log(calcularIvaDeclarada(100)); // funciona: 21  
  
function calcularIvaDeclarada(precio) {  
  return precio * 0.21;  
}
```

```
console.log(calcularIvaExpresion(100)); // ReferenceError: Cannot access
before initialization

const calcularIvaExpresion = function (precio) {
  return precio * 0.21;
};
```

4.2 Parámetros y argumentos

Parámetro es el nombre que recibe un dato dentro de la definición de la función; argumento es el valor concreto que se pasa al llamarla. En function calcularIva(precio), precio es el parámetro; en calcularIva(100), 100 es el argumento.

Parámetros por defecto

Permiten asignar un valor que se usa automáticamente cuando no se pasa ese argumento al llamar a la función.

```
function calcularPrecioConIva(precioBase, tipoIva = 0.21) {
  return Number((precioBase * (1 + tipoIva)).toFixed(2));
}

console.log(calcularPrecioConIva(100)); // 121 (usa el 0.21 por defecto)
console.log(calcularPrecioConIva(100, 0.10)); // 110 (usa el valor indicado)
```

Parámetros *rest*: agrupar argumentos sueltos

Cuando una función debe aceptar un número indeterminado de argumentos, el operador ... delante del último parámetro los agrupa a todos en un único array.

```
function sumarTodos(...numeros) {
  let total = 0;

  for (const numero of numeros) {
    total += numero;
  }

  return total;
}

console.log(sumarTodos(1, 2, 3)); // 6
console.log(sumarTodos(10, 20, 30, 40)); // 100
```

Nota: El array es el tipo de dato pensado precisamente para agrupar varios valores en uno solo; se estudiará más adelante, junto con sus métodos. Por ahora, basta con saber que ...numeros convierte "todos los argumentos que sobren" en una lista que se puede recorrer con for...of.

Paso de argumentos: valor frente a referencia

Ya se explicó que los primitivos se copian por valor y los objetos y arrays se copian por referencia. Esa misma regla se aplica al pasar argumentos a una función.

```
function intentarDuplicar(numero) {
  numero = numero * 2;
}
```

```
let cantidad = 5;
intentarDuplicar(cantidad);
console.log(cantidad); // 5 (no cambia: se pasó una COPIA del valor)

function aplicarDescuento(producto) {
  producto.precio = producto.precio * 0.9; // modifica una propiedad del
  objeto
}

const articulo = { nombre: 'Ratón', precio: 20 };
aplicarDescuento(articulo);
console.log(articulo.precio); // 18 (SÍ cambia: se pasó una referencia al
mismo objeto)
```

Importante: Modificar un objeto recibido como parámetro dentro de una función afecta al objeto original, fuera de la función, porque ambas variables (la del código que llama y el parámetro dentro de la función) apuntan al mismo objeto en memoria. Esto es intencionado y muy útil, pero también una fuente de errores si no se tiene en cuenta: una función que "solo debía leer" un objeto puede acabar modificándolo sin que quien la usa lo espere.

4.3 El valor de retorno

return termina la ejecución de la función de inmediato y devuelve el valor indicado a quien la llamó. Si una función no tiene ningún return, o tiene un return sin valor, devuelve undefined.

```
function registrarVisita() {
  console.log('Visita registrada');
  // no hay return: la función devuelve undefined
}

const resultado = registrarVisita(); // imprime 'Visita registrada'
console.log(resultado); // undefined
```

Retorno anticipado (*early return*)

Un patrón muy habitual en código profesional es salir de la función en cuanto se detecta un caso no válido, en lugar de anidar toda la lógica dentro de un único if. Esto se conoce como *guard clause* (cláusula de guarda).

```
// Con anidamiento: la lógica principal queda "escondida" dentro del if
function procesarPedidoAnidado(pedido) {
  if (pedido.total > 0) {
    if (pedido.stockDisponible) {
      return 'Pedido confirmado';
    } else {
      return 'Sin stock';
    }
  } else {
    return 'Pedido vacío';
  }
}
```

```
// Con guard clauses: cada caso no válido sale de inmediato
function procesarPedido(pedido) {
  if (pedido.total <= 0) {
    return 'Pedido vacío';
  }

  if (!pedido.stockDisponible) {
    return 'Sin stock';
  }

  return 'Pedido confirmado';
}
```

Nota: Las dos funciones anteriores hacen exactamente lo mismo. La segunda versión suele considerarse más legible en equipos profesionales: cada condición de error se resuelve y se aparta de inmediato, dejando el caso "todo va bien" al final, sin anidamiento.

4.4 Arrow functions

Las arrow functions (funciones flecha) son una sintaxis más compacta para escribir function expressions, introducida en ES6. Se usan de forma constante en JavaScript moderno.

```
// Function expression tradicional
const calcularIva = function (precio) {
  return precio * 0.21;
};

// La misma función, como arrow function
const calcularIvaFlecha = (precio) => {
  return precio * 0.21;
};

// Si el cuerpo es una única expresión, se puede omitir "return" y las llaves
const calcularIvaCorta = (precio) => precio * 0.21;
```

Situación	Sintaxis
Un solo parámetro	precio => precio * 0.21 (los paréntesis son opcionales, pero muchas guías de estilo profesionales piden ponerlos siempre: (precio) => ...)
Varios parámetros	(a, b) => a + b (paréntesis obligatorios)
Sin parámetros	() => console.log('hola')
Cuerpo de una sola expresión	Sin llaves ni return: el valor de la expresión se devuelve automáticamente (<i>implicit return</i>)

Situación	Sintaxis
Cuerpo con varias sentencias	Llaves obligatorias, y return explícito si se quiere devolver algo

Nota: En la práctica profesional, las arrow functions se usan sobre todo para funciones cortas y para pasarlas como argumento a otras funciones (algo muy habitual con los métodos de array, y con la gestión de eventos). Las function declarations siguen siendo una opción perfectamente válida, especialmente para funciones con nombre propio que se llaman por sí solas, no como argumento de otra función.

Ejercicios · Estructuras de Control

Ejercicio 1. Escribe una función `clasificarImc(imc)` que reciba un índice de masa corporal (número) y devuelva, usando `if / else if / else`, uno de estos textos: "Bajo peso" (menos de 18.5), "Normal" (entre 18.5 y 24.9), "Sobrepeso" (entre 25 y 29.9) u "Obesidad" (30 o más).

Ejercicio 2. Reescribe la siguiente función usando el operador ternario, de forma que ocupe una sola línea dentro del cuerpo de la función:

```
function obtenerEtiquetaStock(unidades) {
  if (unidades > 0) {
    return 'En stock';
  } else {
    return 'Agotado';
  }
}
```

Ejercicio 3. Escribe una función `nombreDiaSemana(numeroDia)` que reciba un número del 1 al 7 y devuelva, usando `switch`, el nombre del día correspondiente (1 = "Lunes" ... 7 = "Domingo"). Si el número no está entre 1 y 7, debe devolver "Día no válido".

Ejercicio 4. Explica, sin ejecutarlo, qué imprimirá este código y por qué (presta atención a los `break`):

```
function categoria(puntos) {
  let resultado;
  switch (true) {
    case puntos >= 100:
      resultado = 'Oro';
      break;
    case puntos >= 50:
      resultado = 'Plata';
    case puntos >= 0:
      resultado = 'Bronce';
      break;
    default:
      resultado = 'Sin categoría';
  }
  return resultado;
}
console.log(categoria(60));
```

Ejercicio 5. Escribe un bucle for que recorra los números del 1 al 30 y muestre por consola "Fizz" si el número es múltiplo de 3, "Buzz" si es múltiplo de 5, "FizzBuzz" si es múltiplo de ambos, y el propio número en cualquier otro caso.

Ejercicio 6. Identifica el error en el siguiente código y corrígelo:

```
let intentos = 0;
```

```
while (intentos < 3) {
  console.log('Intento fallido');
}
```

Ejercicio 7. Usando for y continue, escribe un bucle que recorra los números del 1 al 20 y sume solo aquellos que sean múltiplos de 3, mostrando el resultado final por consola.

5. El objeto Date

Date representa un instante concreto en el tiempo. A diferencia de Math, sí es un constructor: cada fecha es una nueva instancia, creada con new Date().

Crear una fecha

```
const ahora = new Date(); // fecha y hora actuales
const fechaConcreta = new Date(2026, 8, 11); // 11 de septiembre de 2026
const desdeTexto = new Date('2026-09-11'); // a partir de una cadena en formato ISO
```

- Si no se le pasa ningún parámetro, coge la fecha actual.
- Si se le pasa 1 parámetro, se indica el número de milisegundos que han transcurrido desde el 1 de Enero de 1970.
- También se le puede pasar un solo parámetro indicando una cadena de texto indicando la fecha (aunque la cadena de texto tiene que tener un formato específico).
- Si se le pasa 3 parámetros: (año, mes, día).
- Si se le pasan 6 parámetros: (año, mes, día, hora, minuto, segundo).

Ejemplos:

```
let fecha = new Date(); // "La fecha y hora actuales"
let fecha = new Date(1000000000000); // "Sat Nov 20 2286 18:46:40"
let fecha = new Date(2009, 5, 1); // 1 de Junio de 2009 (00:00:00)
let fecha = new Date(2009, 5, 1, 19, 29, 39); // 1 de Junio de 2009 (19:29:39)
```

Importante: El constructor new Date(año, mes, día) numera los meses empezando en 0: enero es 0 y diciembre es 11. En el ejemplo anterior, 8 corresponde a septiembre, no a agosto. Este es uno de los errores más habituales al trabajar con fechas en JavaScript; usar directamente una cadena en formato ISO ('2026-09-11'), como en la tercera línea, evita este problema por completo.

Obtener componentes de una fecha

Método	Qué devuelve
.getFullYear()	Año (4 dígitos)

Método	Qué devuelve
.getMonth()	Mes, de 0 (enero) a 11 (diciembre)
.getDate()	Día del mes, de 1 a 31
.getDay()	Día de la semana, de 0 (domingo) a 6 (sábado)
.getHours() / .getMinutes()	Hora y minutos

Mostrar una fecha con formato

Para mostrar una fecha al usuario, en vez de construir el texto a mano concatenando los componentes anteriores, se usa `toLocaleDateString()`, que aplica el formato propio de un idioma o región.

```
const fechaPedido = new Date('2026-09-11');

console.log(fechaPedido.toLocaleDateString('es-ES')); // '11/9/2026'
console.log(fechaPedido.toLocaleDateString('es-ES', {
  day: 'numeric',
  month: 'long',
  year: 'numeric'
})); // '11 de septiembre de 2026'
```

Operar con fechas

Para sumar o restar días, se combina un método *getter* con su *setter* equivalente:

```
const fechaPedido = new Date('2026-09-11');
const fechaEntregaEstimada = new Date(fechaPedido);
fechaEntregaEstimada.setDate(fechaPedido.getDate() + 5);

console.log(fechaEntregaEstimada.toLocaleDateString('es-ES')); // '16/9/2026'
```

Para comparar dos fechas, se convierten a su valor numérico interno (milisegundos desde el 1 de enero de 1970) con `getTime()`, y se comparan como números:

```
const fechaLimite = new Date('2026-09-15');
const hoy = new Date();

if (hoy.getTime() > fechaLimite.getTime()) {
  console.log('Plazo vencido');
} else {
  console.log('Todavía en plazo');
}
```

Nota: La API nativa de `Date` es funcional pero verbosa, y algunas operaciones (como calcular la diferencia en días entre dos fechas, o trabajar con zonas horarias) resultan incómodas. En proyectos profesionales es habitual apoyarse en una librería especializada, como *date-fns* o *Day.js*, para ese tipo de cálculos más avanzados.

5.1 JSON: intercambio de datos

JSON (JavaScript Object Notation) es un formato de texto para representar datos estructurados, basado en la sintaxis de los objetos y arrays de JavaScript, pero más estricto: las claves siempre van entre comillas dobles, no admite comas finales, y no puede contener funciones ni el valor undefined. Es el formato más usado para intercambiar datos entre un cliente y un servidor.

JSON.stringify(): de objeto JavaScript a texto

Convierte un objeto o array de JavaScript en una cadena de texto con formato JSON.

```
const producto = {
  nombre: 'Teclado mecánico',
  precio: 79.99,
  disponible: true
};

const textoJson = JSON.stringify(producto);
console.log(textoJson); // '{"nombre":"Teclado
mecánico","precio":79.99,"disponible":true}'
console.log(typeof textoJson); // 'string'
```

JSON.stringify() admite un tercer argumento para dar formato legible al resultado (muy útil al mostrarlo por consola mientras se depura):

```
console.log(JSON.stringify(producto, null, 2));
/*
{
  "nombre": "Teclado mecánico",
  "precio": 79.99,
  "disponible": true
}
*/
```

JSON.parse(): de texto a objeto JavaScript

Hace el proceso inverso: convierte una cadena de texto con formato JSON en un objeto o array de JavaScript, ya utilizable con la sintaxis habitual (objeto.propiedad).

```
const textoRecibido = '{"nombre":"Teclado
mecánico","precio":79.99,"disponible":true}';
const productoRecuperado = JSON.parse(textoRecibido);

console.log(productoRecuperado.nombre); // 'Teclado mecánico'
console.log(typeof productoRecuperado); // 'object'
```

Importante: Si el texto que se pasa a JSON.parse() no es JSON válido (por ejemplo, si le faltan comillas a una clave, o tiene una coma sobrante), la función lanza un error y detiene la ejecución del script en ese punto. La forma correcta de protegerse frente a esto es capturar el error con try / catch, una herramienta que se estudiará más adelante, junto con la comunicación asíncrona con el servidor (que es, precisamente, el contexto donde más se usa JSON.parse() en aplicaciones reales).

Nota: localStorage y las cookies, que se verán en el próximo tema, solo pueden almacenar texto. Cuando se necesita guardar un objeto o un array completo, el patrón

habitual es convertirlo a texto con `JSON.stringify()` antes de guardarlo, y volver a convertirlo con `JSON.parse()` al recuperarlo.

Ejercicios

Ejercicio 1. Escribe una función `normalizarNombreUsuario(texto)` que reciba una cadena con posibles espacios al principio o al final, y devuelva el texto sin esos espacios y con la primera letra en mayúscula (por ejemplo, ' ana ' debe devolver 'Ana'). Pista: puedes combinar `.trim()`, `.slice()` y `.toUpperCase()`.

Ejercicio 2. Escribe una función `generarCodigoDescuento()` que devuelva una cadena con el formato DESC- seguido de un número entero aleatorio de 4 cifras (entre 1000 y 9999).

Ejercicio 3. Escribe una función `diasHastaFecha(fechaObjetivo)` que reciba una fecha en formato 'YYYY-MM-DD' y devuelva el número de días que faltan desde hoy hasta esa fecha (puedes usar `getTime()` y dividir la diferencia en milisegundos entre $1000 * 60 * 24$).

Ejercicio 4. Dado el siguiente objeto, escribe el código para convertirlo a una cadena JSON legible (con sangrado), y después vuelve a convertir esa cadena en un objeto JavaScript, mostrando por consola el valor de stock del objeto recuperado:

```
const articulo = { sku: 'TC-2201', stock: 15, activo: true };
```

6. Transformar arrays: map, filter y reduce

Estos tres métodos son, probablemente, los más usados de todo JavaScript moderno. Los tres reciben un callback, y ninguno modifica el array original: todos devuelven un resultado nuevo.

map(): transforma cada elemento

Devuelve un array nuevo, de la misma longitud que el original, con el resultado de aplicar el callback a cada elemento.

```
const precios = [19.99, 45.5, 12.0];
const preciosConIva = precios.map(precio => Number((precio *
1.21).toFixed(2)));

console.log(preciosConIva); // [24.19, 55.06, 14.52]
console.log(precios); // [19.99, 45.5, 12] (sin modificar)
```

filter(): selecciona elementos

Devuelve un array nuevo, con solo los elementos para los que el *callback* devuelve true. Puede tener una longitud menor (o igual) que el original.

```
const inventario = [
  { nombre: 'Teclado', stock: 12 },
  { nombre: 'Ratón', stock: 0 },
  { nombre: 'Monitor', stock: 5 }
];

const disponibles = inventario.filter(articulo => articulo.stock > 0);
```

```
console.log(disponibles); // [{ nombre: 'Teclado', ... }, { nombre: 'Monitor', ... }]
```

reduce(): acumula un único resultado

reduce() es el más flexible, y también el que más cuesta interiorizar al principio. Recorre el array acumulando un resultado en una variable (el *acumulador*), que va pasando de una iteración a la siguiente.

```
const precios = [19.99, 45.5, 12.0];

const total = precios.reduce((acumulador, precioActual) => {
  return acumulador + precioActual;
}, 0); // el 0 final es el valor inicial del acumulador

console.log(total); // 77.49
```

Es útil seguir la ejecución paso a paso la primera vez que se usa:

Iteración	acumulador (antes)	precioActual	acumulador (después)
1	0 (valor inicial)	19.99	19.99
2	19.99	45.5	65.49
3	65.49	12.0	77.49

Importante: El segundo argumento de reduce() (el valor inicial del acumulador) casi nunca debería omitirse. Si se omite, reduce() toma el primer elemento del array como valor inicial y empieza a iterar desde el segundo, lo cual puede dar resultados inesperados (o un error, si el array está vacío). Indicar siempre el valor inicial explícitamente hace el comportamiento predecible en cualquier caso.

Encadenar los tres métodos

Como map() y filter() devuelven un array nuevo, se pueden encadenar entre sí y terminar con un reduce(), formando una pequeña "cadena de montaje" de datos.

```
const inventario = [
  { nombre: 'Teclado', precio: 45, stock: 12 },
  { nombre: 'Ratón', precio: 20, stock: 0 },
  { nombre: 'Monitor', precio: 150, stock: 5 }
];

const valorTotalDisponible = inventario
  .filter(articulo => articulo.stock > 0)
  .map(articulo => articulo.precio * articulo.stock)
  .reduce((total, valorArticulo) => total + valorArticulo, 0);

console.log(valorTotalDisponible); // 45*12 + 150*5 = 1290
```

Nota: Este estilo (encadenar filter, map y reduce en lugar de escribir un único bucle for con varias condiciones dentro) se conoce como programación funcional, y es el estándar en JavaScript profesional moderno: cada método hace una sola cosa, el código se lee

casi como una frase ("filtra los disponibles, calcula su valor, súmalo todo"), y ninguno modifica los datos originales.

6.1 Ordenar y combinar arrays

sort(): cuidado con el orden por defecto

sort() ordena el array modificándolo directamente (a diferencia de map, filter y reduce). Por defecto, convierte todos los elementos a texto y los ordena alfabéticamente, incluso si son números.

```
const cantidades = [10, 1, 21, 2];

console.log(cantidades.sort()); // [1, 10, 2, 21] (orden alfabético, no numérico!)
```

Importante: Para ordenar números correctamente, hay que pasarle a sort() una función de comparación: recibe dos elementos (a y b) y debe devolver un número negativo si a va antes que b, positivo si va después, o 0 si son iguales. Restar ambos valores resuelve el caso numérico de forma directa.

```
const cantidades = [10, 1, 21, 2];

console.log(cantidades.sort((a, b) => a - b)); // [1, 2, 10, 21] (ascendente)
console.log(cantidades.sort((a, b) => b - a)); // [21, 10, 2, 1] (descendente)
```

Combinar arrays: concat() y el operador spread

El operador ..., usado para *agrupar* varios argumentos en un array (parámetros *rest*), tiene el efecto contrario cuando se usa dentro de un array literal: expande un array existente en sus elementos individuales.

```
const categoriaOficina = ['Teclado', 'Ratón'];
const categoriaPantallas = ['Monitor', 'Webcam'];

// Con concat()
const catalogoCompleto1 = categoriaOficina.concat(categoriaPantallas);

// Con el operador spread (forma preferida en código moderno)
const catalogoCompleto2 = [...categoriaOficina, ...categoriaPantallas];

console.log(catalogoCompleto2); // ['Teclado', 'Ratón', 'Monitor', 'Webcam']
```

El mismo operador permite crear una copia independiente de un array, sin compartir referencia con el original.

```
const original = [1, 2, 3];
const copia = [...original];

copia.push(4);
console.log(original); // [1, 2, 3] (no afectado)
console.log(copia); // [1, 2, 3, 4]
```

Nota: Esto último es importante: const copia = original; (sin ...) NO crea una copia, sino una segunda variable apuntando al mismo array, exactamente igual que ocurría con los

objetos. El operador spread es una de las formas más habituales de crear una copia real en JavaScript moderno.

6.2 Desestructuración de arrays

La desestructuración permite extraer varios elementos de un array y asignarlos a variables individuales, en una sola línea, en lugar de acceder a cada posición por separado.

```
const coordenadas = [40.4168, -3.7038];

// Sin desestructurar
const latitud = coordenadas[0];
const longitud = coordenadas[1];

// Con desestructuración: mismo resultado, una sola línea
const [lat, lon] = coordenadas;
console.log(lat, lon); // 40.4168 -3.7038
```

Se pueden omitir posiciones que no interesen, dejando una coma vacía, y asignar valores por defecto para posiciones que podrían no existir:

```
const fila = ['Ana', 28, 'Madrid'];
const [nombre, , ciudad] = fila; // se salta la edad, en el índice 1
console.log(nombre, ciudad); // 'Ana' 'Madrid'

const [primero, segundo = 'Sin definir'] = ['único valor'];
console.log(segundo); // 'Sin definir' (no había un segundo elemento)
```

Combinada con el operador rest, permite separar los primeros elementos del resto:

```
const clasificacion = ['Oro', 'Plata', 'Bronce', 'Cuarto', 'Quinto'];
const [primerPuesto, segundoPuesto, ...resto] = clasificacion;

console.log(primerPuesto); // 'Oro'
console.log(resto); // ['Bronce', 'Cuarto', 'Quinto']
```

7. BOM: objetos del navegador

Además de los objetos nativos del propio lenguaje, el navegador expone su propio conjunto de objetos para interactuar con la ventana, el dispositivo y la navegación: el BOM (Browser Object Model).

7.1 El objeto window

window representa la ventana o pestaña del navegador, y es el objeto global de todo el código que se ejecuta en el cliente. Esto tiene una consecuencia importante: todo lo que hasta ahora se ha usado sin ningún prefijo (document, console, alert, setTimeout) es, en realidad, una propiedad de window.

```
console.log(document === window.document); // true
console.log(alert === window.alert); // true

// Estas dos líneas hacen exactamente lo mismo
console.log('Hola');
```

```
window.console.log('Hola');
```

Tamaño de la Ventana

```
console.log(window.innerWidth); // ancho del área visible, en píxeles
console.log(window.innerHeight); // alto del área visible, en píxeles
```

Temporizadores: setTimeout y setInterval

window también proporciona funciones para ejecutar código pasado un tiempo determinado, sin bloquear el resto de la página mientras se espera.

```
// Ejecuta la función UNA vez, transcurridos 3000 ms (3 segundos)
const idTimeout = setTimeout(() => {
  console.log('Han pasado 3 segundos');
}, 3000);

// Ejecuta la función CADA 2000 ms, de forma repetida
const idInterval = setInterval(() => {
  console.log('Tic');
}, 2000);

// Cancela un temporizador antes de que se dispare (o antes de la siguiente
repetición)
clearTimeout(idTimeout);
clearInterval(idInterval);
```

Importante: setTimeout y setInterval no detienen la ejecución del resto del script mientras esperan: el código que va después se ejecuta de inmediato, y la función indicada se ejecuta más tarde, cuando le toque. Este comportamiento (no bloqueante) es la base de la programación asíncrona en JavaScript.

7.2 navigator: información del navegador

navigator expone información sobre el navegador y, en algunos casos, sobre el dispositivo del usuario.

Propiedad	Qué devuelve
navigator.language	Idioma configurado en el navegador (por ejemplo, 'es-ES')
navigator.onLine	true o false según si el dispositivo tiene conexión a red
navigator.userAgent	Cadena de texto que identifica navegador y sistema operativo
navigator.geolocation	API para solicitar la ubicación del usuario (requiere su permiso explícito)

```
if (navigator.onLine) {
  console.log('Conectado a internet');
} else {
  console.log('Sin conexión: mostrando datos guardados localmente');
```

```
}
```

Importante: `navigator.userAgent` se usaba históricamente para detectar qué navegador visitaba la página y adaptar el comportamiento en consecuencia. Hoy se considera mala práctica: el valor puede ser falsificado con facilidad, varía mucho entre versiones, y su formato es, por razones históricas, poco fiable. La alternativa profesional es la detección de características (comprobar directamente si el navegador soporta una función concreta, en lugar de intentar adivinarlo por su identidad), un enfoque que se verá más adelante, al comprobar el soporte de almacenamiento local.

7.3 Navegar a otra URL

```
location.href = 'https://tienda.com/carrito'; // navega a otra URL, dejando
rastro en el historial
location.reload(); // recarga la página actual
```

7.4 history: el historial de navegación

history permite moverse por el historial de páginas visitadas en la sesión actual del navegador, sin conocer las URL concretas.

```
history.back(); // equivale a pulsar el botón "atrás" del navegador
history.forward(); // equivale a pulsar el botón "adelante"
history.go(-2); // retrocede dos páginas en el historial

console.log(history.length); // número de entradas en el historial de la
pestaña actual
```

7.5 Ventanas: apertura, cierre y comunicación

Abrir y cerrar ventanas

window.open() abre una nueva ventana o pestaña, y devuelve una referencia a ella que permite seguir controlándola desde el script que la abrió.

```
const ventanaAyuda = window.open(
  'https://tienda.com/ayuda',
  'ventanaAyuda',
  'width=500,height=600'
);

// Más adelante, desde el script que la abrió:
ventanaAyuda.close();
```

Importante: La mayoría de navegadores bloquean por defecto las ventanas abiertas con `window.open()` si no son consecuencia directa de una acción del usuario (por ejemplo, un clic). Abrir una ventana automáticamente al cargar la página, sin que medie una interacción, suele ser bloqueado por el propio navegador como si fuera una ventana emergente no deseada (pop-up).

8. Cookies y almacenamiento cliente

Hay tres formas de conservar información en el navegador entre visitas o entre páginas: cookies, `localStorage` y `sessionStorage`, cada una con su propio propósito.

8.1 Cookies: qué son y para qué sirven

Una cookie es un pequeño fragmento de texto (con un límite de, aproximadamente, 4 KB) que el navegador guarda asociado a un dominio concreto. Su característica más particular es que el navegador la envía automáticamente al servidor en cada petición HTTP a ese dominio, sin que el código JavaScript tenga que hacer nada especial para ello. Por eso, históricamente, las cookies se han usado para mantener la sesión de un usuario iniciada entre distintas páginas.

Crear una cookie

El acceso a las cookies desde JavaScript se hace a través de `document.cookie`, una propiedad con un funcionamiento poco habitual: escribir en ella no borra las cookies existentes, sino que añade o actualiza una.

```
document.cookie = 'tema=oscuro; max-age=2592000; path=/';
```

Parte	Significado
tema=oscuro	El par clave-valor que se quiere guardar
max-age=2592000	Segundos hasta que caduque (en el ejemplo, 30 días); sin esta parte, la cookie desaparece al cerrar el navegador
path=/	Ruta del sitio en la que la cookie está disponible (/ significa "todo el dominio")

Leer las cookies

`document.cookie`, al leerlo, no devuelve un objeto ni una cookie concreta: devuelve **todas** las cookies del dominio en una única cadena de texto, separadas por `;`. Para obtener el valor de una cookie concreta, hay que buscarla dentro de esa cadena.

```
console.log(document.cookie); // 'tema=oscuro; idioma=es; usuario=ana'

/**
 * Busca el valor de una cookie concreta por su nombre.
 * @param {string} nombre - Nombre de la cookie a buscar.
 * @returns {string|null} Valor de la cookie, o null si no existe.
 */
function obtenerCookie(nombre) {
  const cookies = document.cookie.split('; ');

  for (const cookie of cookies) {
    const [clave, valor] = cookie.split('=');
    if (clave === nombre) {
      return valor;
    }
  }

  return null;
}

console.log(obtenerCookie('tema')); // 'oscuro'
```

Eliminar una cookie

No existe un método directo para borrar una cookie. La forma habitual es volver a crearla con el mismo nombre y una fecha de caducidad ya pasada, lo que hace que el navegador la elimine de inmediato.

```
document.cookie = 'tema=; max-age=0; path=/';
```

Importante: Como se explicó en el Tema 1, todo el código JavaScript del cliente es visible y manipulable por el usuario, y las cookies no son una excepción: pueden leerse y modificarse desde las DevTools con la misma facilidad que cualquier otro dato. Por eso, información sensible como contraseñas o tokens de sesión no debería depender únicamente de una cookie creada desde JavaScript. Las cookies verdaderamente seguras para sesiones de usuario se crean desde el servidor con el atributo `HttpOnly`, que las hace invisibles para `document.cookie` y solo accesibles por el propio servidor; esa parte corresponde al desarrollo del lado del servidor, fuera del alcance de este módulo.

8.2 localStorage: almacenamiento persistente

`localStorage` es una API pensada específicamente para guardar datos en el navegador, con una diferencia clave respecto a las cookies: nunca se envía al servidor. Los datos quedan exclusivamente en el navegador del usuario, lo que la hace más rápida (no añade peso a cada petición HTTP) y con mucha más capacidad (varios megabytes, frente a los 4 KB de una cookie).

La API: `setItem`, `getItem`, `removeItem`, `clear`

```
localStorage.setItem('tema', 'oscuro'); // guarda un valor
console.log(localStorage.getItem('tema')); // 'oscuro' (recupera un valor)
localStorage.removeItem('tema'); // elimina esa clave concreta
localStorage.clear(); // elimina TODAS las claves guardadas por este sitio
```

Importante: `localStorage` solo almacena texto (string). Si se guarda un valor de otro tipo, JavaScript lo convierte a texto de forma implícita, con resultados poco útiles en el caso de objetos y arrays: `localStorage.setItem('usuario', { nombre: 'Ana' })` guarda literalmente el texto `'[object Object]'`, no los datos del objeto. Para guardar un objeto o un array completo, hay que convertirlo primero con `JSON.stringify()`, y volver a convertirlo con `JSON.parse()` al leerlo.

```
const preferencias = { tema: 'oscuro', idioma: 'es' };

localStorage.setItem('preferencias', JSON.stringify(preferencias));

const preferenciasGuardadas=JSON.parse(localStorage.getItem('preferencias'));
console.log(preferenciasGuardadas.idioma); // 'es'
```

Los datos guardados en `localStorage` persisten indefinidamente: no caducan por sí solos, y siguen disponibles aunque se cierre el navegador o se reinicie el ordenador. Solo desaparecen si el usuario los borra manualmente (o el propio sitio los elimina con código).

8.3 sessionStorage: almacenamiento por pestaña

sessionStorage ofrece exactamente la misma API que localStorage (setItem, getItem, removeItem, clear), pero con un ciclo de vida distinto: los datos solo existen mientras esa pestaña del navegador permanece abierta. Al cerrarla, desaparecen.

```
sessionStorage.setItem('pasoActual', '2');
console.log(sessionStorage.getItem('pasoActual')); // '2'
```

Nota: Un uso típico de sessionStorage es conservar el progreso de un formulario dividido en varios pasos (datos personales, dirección, pago), de forma que si el usuario recarga la página por accidente no pierda lo ya introducido, pero sin que ese progreso persista indefinidamente ni se comparta con otras pestañas donde el usuario pueda tener otro proceso de compra distinto en marcha.

Importante: Aunque el nombre pueda sugerir "toda la sesión del usuario", sessionStorage está aislado por pestaña, no por usuario ni por sitio. Si se abren dos pestañas con la misma página, cada una tiene su propio sessionStorage independiente, aunque estén mostrando la misma URL al mismo tiempo.

8.4 Comparativa: cookies, localStorage y sessionStorage

	Cookies	localStorage	sessionStorage
Capacidad aproximada	4 KB	5-10 MB	5-10 MB
¿Se envía al servidor?	Sí, en cada petición	No	No
Caducidad	La que se configure	Nunca (hasta que se borre)	Al cerrar la pestaña
Ámbito	Dominio (y ruta, opcionalmente)	Dominio	Pestaña
Tipo de dato	Texto	Texto	Texto
Uso típico	Sesión de servidor, preferencias que el backend necesita leer	Preferencias de la interfaz, datos de uso frecuente sin caducidad	Progreso temporal dentro de una única visita

Nota: Como regla general: si el servidor necesita ver ese dato en cada petición, se usa una cookie. Si el dato es solo para el propio código del cliente (una preferencia de interfaz, un borrador, un identificador temporal), localStorage o sessionStorage son la opción más adecuada, precisamente porque no viajan en cada petición HTTP y no añaden peso innecesario a la comunicación con el servidor.

8.5 Comprobar la disponibilidad de almacenamiento

Como se comentó en el apartado anterior al hablar de navigator, la forma profesional de saber si una funcionalidad está disponible es comprobarlo directamente, en lugar de asumirlo. Aunque hoy localStorage y sessionStorage están soportados por todos los navegadores modernos, algunos entornos (ciertos modos de navegación privada, restricciones de política de la empresa, o el propio almacenamiento lleno) pueden hacer que no estén disponibles o que las operaciones fallen.

```
if (typeof Storage !== 'undefined') {  
  console.log('localStorage y sessionStorage están disponibles');  
} else {  
  console.log('Este navegador no soporta Web Storage');  
}
```

Añadir a POO

const con objetos y arrays: una confusión habitual

Un error muy común al empezar es pensar que const hace que un objeto o un array sea inmutable. Esto es falso: const impide **reasignar la variable**, pero si el valor es un objeto o un array, su **contenido interno sí puede modificarse**.

```
const usuario = { nombre: 'Laura', rol: 'admin' };  
  
usuario.rol = 'editor'; // Modificamos una propiedad, no reasignamos la  
variable  
console.log(usuario); // { nombre: 'Laura', rol: 'editor' }  
  
usuario = { nombre: 'Marta' }; // TypeError: Assignment to constant variable.
```

```
const idsSeleccionados = [1, 2, 3];  
  
idsSeleccionados.push(4); // Modificamos el array, no reasignamos la variable  
console.log(idsSeleccionados); // [1, 2, 3, 4]  
  
idsSeleccionados = []; // TypeError: Assignment to constant variable.
```

Nota: Si se necesita que un objeto sea realmente inmutable (que ni siquiera sus propiedades puedan cambiar), const no es suficiente: haría falta Object.freeze(). Se tratará con más detalle en el tema de objetos.

Tipos por referencia

Un valor por referencia (objetos, arrays, funciones) se almacena en memoria y la variable guarda una **referencia** (dirección) a ese valor, no el valor en sí. Esto tiene consecuencias importantes al copiar o comparar.

```
const objetoA = { color: 'rojo' };
```

```
const objetoB = objetoA; // objetoB apunta al MISMO objeto en memoria, no es
una copia

objetoB.color = 'azul';
console.log(objetoA.color); // 'azul', también cambió objetoA

console.log(objetoA === objetoB); // true, comparan la misma referencia
console.log({ color: 'azul' } === { color: 'azul' }); // false, son objetos
distintos en memoria
```

Importante: La diferencia entre "copiar por valor" (primitivos) y "copiar por referencia" (objetos y arrays) es una de las fuentes de errores más comunes en JavaScript, especialmente al trabajar con el estado de una aplicación.